

Gradient de la politique

(Policy Gradient)

Marin Ferecatu

Notes de cours

UE RCP211, Séance RL No. 4

CNAM Paris

Document rédigé en L^AT_EX

22 avril 2026

Table des matières

Avant-propos	2
Notations globales	3
1 Introduction	4
1.1 Le cadre général. Le problème qu'on essaye de résoudre	4
1.2 Motivation. Transition depuis la VFA et les réseaux DQN	5
1.3 Plan du document	6
1.4 Exemples motivationnels : chifoumi, gridworld avec alias	7
2 Différences finies	9
2.1 Quel objectif $J(\theta)$ maximiser ?	9
2.2 Principe général : ascension de gradient	10
2.3 Principe général des différences finies	11
2.4 Application à une politique paramétrée	11
2.5 Exemple : AIBO	12
2.6 Pourquoi cette approche est-elle peu satisfaisante en RL ?	14
3 Gradient de la politique	16
3.1 Fonction de score	16
3.2 Gradient de la politique	19
3.3 Monte Carlo Policy Gradient	22
3.4 Actor Critic Policy Gradient	25
3.5 Variantes / Extensions	32
Conclusion	36
Bibliographie	37

Avant-propos

Ces notes poursuivent directement les trois polycopiés précédents : *Apprentissage par renforcement : cadre général et programmation dynamique* [1], *Apprentissage par renforcement : prédiction et contrôle Model Free* [2] et *Approximation de la fonction de valeur et réseaux DQN* [3]. On y suppose donc connus le cadre markovien, la définition d’une politique et des fonctions de valeur [1, section 2.4, p. 12 et sections 2.6–2.7, p. 13], les équations de Bellman [1, section 2.10, p. 15], ainsi que les principales méthodes model-free fondées sur les valeurs : Monte Carlo, TD, SARSA, Q-learning et DQN [2, chapitres 2 et 3], [3, chapitres 2 et 3].

Le troisième polycopié était centré sur l’approximation de la fonction de valeur et sur les méthodes de type DQN, c’est-à-dire sur une famille de méthodes où l’on apprend principalement une quantité du type $q(s, a)$, puis où l’on choisit ensuite une action gloutonne ou presque gloutonne. Ici, on change de point de vue. Au lieu d’apprendre d’abord une valeur puis d’en déduire une politique, on va apprendre *directement une politique paramétrée*. C’est cette idée qui mène aux méthodes de *gradient de la politique* (*policy gradient methods*).

La transition conceptuelle est importante. Dans RL3, les équations de Bellman restaient la boussole théorique, mais la quantité représentée explicitement était une fonction de valeur approchée [3, section 3.2, p. 17]. Dans ce nouveau document, l’objet central devient une loi conditionnelle $a \mapsto \pi_\theta(a | s)$ dépendant d’un vecteur de paramètres θ . Le but n’est plus seulement d’évaluer correctement les actions ; il est de modifier les paramètres de façon à augmenter l’espérance du retour.

Notations globales

Les notations des premiers polycopiés sont conservées. On ajoute ici quelques notations spécifiques au gradient de la politique.

$\pi_{\theta}(a \mid s)$	politique paramétrée par le vecteur θ ;
$\theta \in \mathbb{R}^d$	vecteur de paramètres de la politique ;
$J(\theta)$	fonction objectif à maximiser ;
τ	trajectoire observée, par exemple $(s_0, a_0, r_1, \dots, s_T)$;
$p_{\theta}(\tau)$	probabilité ou densité associée à la trajectoire τ ;
$\psi_{\theta}(s, a)$	vecteur score de la politique, noté souvent $\nabla_{\theta} \log \pi_{\theta}(a \mid s)$;
g_t	retour tronqué ou <i>return-to-go</i> à partir de l'instant t ;
$A_{\pi}(s, a)$	avantage (<i>advantage</i>) de l'action a dans l'état s ;
$\hat{A}_t$	estimation de l'avantage au temps t ;
ρ	distribution initiale sur les états ;
μ_{π}	mesure d'occupation des états sous la politique π .

Comme dans les documents précédents, les majuscules S_t, A_t, R_{t+1} désignent des variables aléatoires, les minuscules s_t, a_t, r_{t+1} désignent des réalisations particulières. Lorsqu'on écrira une moyenne empirique sur un batch de trajectoires, les trajectoires observées seront notées en minuscules, par exemple $\tau^{(1)}, \dots, \tau^{(m)}$.

Chapitre 1

Introduction

1.1 Le cadre général. Le problème qu'on essaye de résoudre

Dans les trois premiers polycopiés, le problème de fond était toujours le même : construire un bon comportement dans un processus de décision markovien afin de maximiser une somme de récompenses futures. Ce but ne change pas ici. Ce qui change, c'est l'objet que l'on choisit d'apprendre directement.

Dans RL2, puis dans RL3, l'idée centrale était d'apprendre une fonction de valeur d'état ou d'état-action, d'abord sous forme tabulaire, puis sous forme approchée par une famille paramétrée [2, chapitre 3], [3, chapitre 2]. Ce passage à l'approximation de fonction permettait déjà de franchir une étape importante : on ne mémorise plus une valeur séparée pour chaque état ou pour chaque couple état-action, et l'on peut au contraire mutualiser l'information entre situations semblables. C'est précisément ce qui rend possible une certaine généralisation lorsque l'espace d'états devient trop grand pour une représentation tabulaire.

Cependant, dans toutes ces approches, la politique reste obtenue de manière *indirecte*. On commence par estimer une quantité de type v_π , q_π ou q_* , puis l'on transforme cette estimation en règle de décision. Dans le cas du contrôle, cela conduit très souvent à une politique gloutonne ou ε -gloutonne par rapport à une valeur d'action estimée. Autrement dit, même lorsque la représentation de q est très puissante — par exemple un réseau de neurones profond dans DQN — la logique reste fondamentalement la même : on apprend d'abord des valeurs, puis on choisit les actions à partir d'elles.

Cette stratégie est extrêmement naturelle et souvent très efficace, mais elle possède aussi des limites structurelles. D'abord, si l'ensemble des actions est continu, très grand, ou de nature combinatoire, la règle

$$a_t \in \operatorname{argmax}_{a \in \mathcal{A}} q(s_t, a)$$

cesse d'être une simple comparaison entre quelques nombres. Elle devient elle-même un problème d'optimisation interne, parfois difficile, coûteux, voire intractable en pratique. Ensuite, même lorsque l'ensemble des actions est discret, la politique gloutonne n'est pas toujours la bonne réponse conceptuelle. Dans certaines situations, on souhaite au contraire apprendre une politique authentiquement stochastique, et pas seulement ajouter un bruit d'exploration artificiel autour d'une règle gloutonne. C'est le cas lorsque plusieurs actions sont presque équivalentes, lorsque l'aléa fait partie d'une bonne stratégie, ou lorsque l'on veut éviter de ramener tout le problème à la recherche systématique d'un *argmax*.

C'est ici qu'interviennent les méthodes de gradient de la politique (*policy gradient methods*). Leur idée centrale est de ne plus passer par le détour consistant à apprendre d'abord une fonction de valeur pour ensuite en extraire une politique. On choisit directement une famille paramétrée de politiques

$$\{\pi_\theta : \theta \in \mathbb{R}^d\},$$

puis l'on cherche un vecteur de paramètres θ qui rende cette politique aussi performante que possible.

Le problème devient alors un problème d'optimisation continue :

$$\max_{\theta \in \mathbb{R}^d} J(\theta),$$

où $J(\theta)$ désigne la performance moyenne de la politique π_θ , c'est-à-dire en pratique l'espérance du retour sous cette politique.

Le mot *gradient* apparaît parce que, lorsque la fonction objectif J est dérivable par rapport aux paramètres, on peut chercher à l'améliorer par des mises à jour de type ascension de gradient :

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_\theta J(\theta_k).$$

Le point important est donc le suivant : on ne cherche plus d'abord à estimer une fonction de valeur pour fabriquer ensuite une politique gloutonne ; on optimise directement une politique paramétrée en augmentant sa performance moyenne.

Cette différence de point de vue est conceptuellement majeure. Dans les approches fondées sur les valeurs, la politique est un objet dérivé. Dans les approches de gradient de la politique, la politique devient l'objet principal de l'apprentissage. C'est ce changement qui permet de traiter plus naturellement les actions continues, les espaces d'actions très riches, et les politiques stochastiques apprises pour elles-mêmes plutôt que construites a posteriori à partir d'une estimation de q .

1.2 Motivation. Transition depuis la VFA et les réseaux DQN

Le passage des méthodes basées sur la valeur vers les méthodes basées sur la politique se comprend bien en regardant ce que savent faire les unes et les autres, et surtout ce qu'elles savent moins bien faire.

Les méthodes *value-based* apprennent une fonction de valeur ou de valeur d'action. Dans leur forme la plus classique, elles conviennent naturellement à des actions discrètes, car la politique est ensuite obtenue par comparaison explicite de quelques valeurs. C'est l'esprit de Q-learning et de DQN. Mais lorsque l'espace d'actions devient continu, ou simplement très grand, l'étape

$$\operatorname{argmax}_a q(s, a)$$

peut devenir coûteuse ou peu praticable. On peut évidemment essayer de l'approcher, mais la difficulté n'a pas disparu : elle a simplement été déplacée.

Les méthodes *policy-based*, au contraire, apprennent directement une loi sur les actions. Si l'action est continue, on peut, par exemple, faire sortir d'un réseau de neurones les paramètres d'une loi gaussienne. Si l'action est discrète, le réseau peut faire sortir des probabilités par une couche *softmax*. Dans les deux cas, la politique est l'objet appris ; il n'est plus nécessaire de reconstruire une règle de décision à partir d'une valeur d'action.

Une deuxième motivation concerne la stochasticité. Une politique paramétrée peut être naturellement stochastique. Cela n'est pas seulement commode pour l'exploration ; cela peut être structurellement nécessaire. Dans certains problèmes, la meilleure stratégie n'est pas déterministe. Nous le verrons sur l'exemple du chifoumi à la section 1.4.

Une troisième motivation concerne la continuité conceptuelle avec RL3. Dans RL3, le message principal était le suivant : lorsqu'une table n'est plus adaptée, on remplace cette table par une fonction paramétrée et l'on ajuste les paramètres par descente de gradient stochastique [3, section 2.3, p. 6]. Le gradient de la politique prolonge exactement cette idée, mais l'objet paramétré n'est plus une approximation de q ; c'est la politique elle-même.

La figure 1.1 donne une vue d'ensemble souvent utilisée pour situer les grandes familles de méthodes. Elle n'est pas une classification parfaite, mais elle est utile pédagogiquement. Elle rappelle que plusieurs

axes conceptuels se croisent : avec modèle ou sans modèle, basé sur une fonction de valeur ou sur une politique, et présence ou non d'un critique. Les méthodes *actor-critic* apparaissent naturellement dans la zone d'intersection : elles possèdent un acteur, c'est-à-dire une politique paramétrée, et un critique, c'est-à-dire un estimateur de valeur. C'est aussi pourquoi des algorithmes modernes comme PPO [20] sont à la fois *policy-based*, *model-free* et *actor-critic*, plutôt que rangés dans une seule case simple.

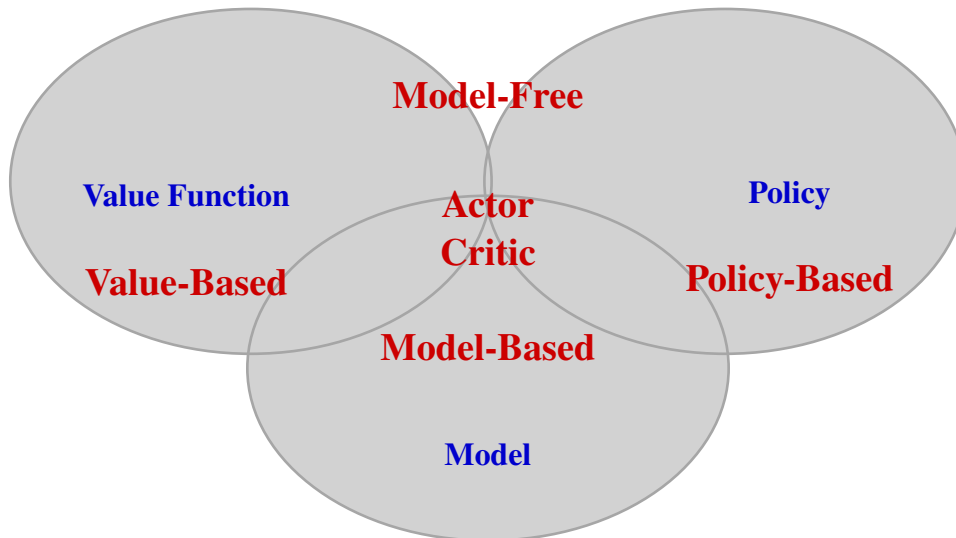


FIGURE 1.1 – Carte conceptuelle simplifiée des grandes familles de méthodes de RL. L'idée essentielle pour ce cours est la transition depuis la zone *value-based*, *model-free* étudiée avec DQN vers la zone *policy-based*, *model-free*, souvent avec critique.

En résumé, on peut voir la transition comme un déplacement de point de vue :

- les méthodes à base de valeur apprennent d'abord *combien vaut* une action ;
- les méthodes à base de politique apprennent directement *comment agir* ;
- les méthodes acteur-critique apprennent les deux objets en parallèle, le critique servant de guide à l'acteur.

1.3 Plan du document

Le chapitre 2 commence volontairement par une idée très naïve : si l'on a une politique paramétrée, on pourrait essayer d'approximer le gradient de la performance par différences finies. Cette approche a l'avantage d'être immédiate, mais elle devient vite coûteuse et bruitée en RL. Elle sert donc surtout d'étape pédagogique.

Le chapitre 3 constitue le coeur du polycopié. On y introduit d'abord la fonction de score et l'astuce du *likelihood ratio gradient*, qui permet de transformer un gradient d'espérance en une espérance contenant un gradient. On l'applique ensuite au cas des trajectoires de RL, ce qui conduit à la formule générale du gradient de la politique. On étudie alors successivement la version Monte Carlo, connue sous le nom de REINFORCE, puis les méthodes *actor-critic*, qui remplacent une partie du retour Monte Carlo par un critique afin de réduire la variance.

Le chapitre 4 est un complément. Il ne revient pas sur toute la programmation dynamique étudiée dans RL1 [1, chapitre 3, p. 21 à 29], mais il montre comment des idées de planification dans un modèle, en particulier l'optimisation de trajectoire, se situent à côté des méthodes de gradient de politique. Cela permet de replacer les approches modernes dans un paysage plus large.

1.4 Exemples motivationnels : chifoumi, gridworld avec alias

Exemple 1 : le chifoumi

Le chifoumi est un exemple très simple, mais conceptuellement très fort. Chaque joueur choisit parmi trois actions : pierre, feuille ou ciseaux. Si les deux joueurs sont rationnels et connaissent parfaitement les règles, une politique déterministe n'est pas stable. Si un joueur choisit toujours pierre, l'autre répondra toujours feuille. Si un joueur choisit toujours feuille, l'autre répondra toujours ciseaux, et ainsi de suite.

La bonne notion de solution n'est donc pas une action fixe, mais une *stratégie mixte*, c'est-à-dire une loi de probabilité sur les actions. Ici, la politique naturelle est

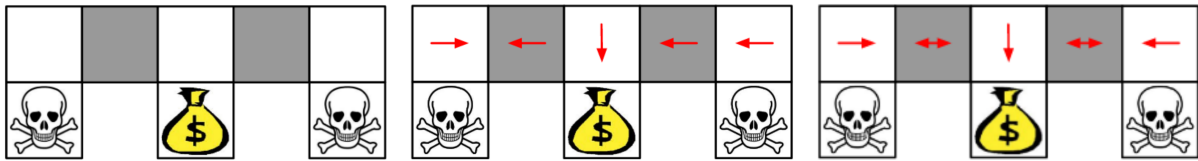
$$\pi(\text{pierre}) = \pi(\text{feuille}) = \pi(\text{ciseaux}) = \frac{1}{3}.$$

Le point important n'est pas le calcul exact de l'équilibre, mais l'idée suivante : dans certains problèmes, la meilleure politique est intrinsèquement stochastique. Un cadre qui apprend directement une distribution sur les actions est alors plus naturel qu'un cadre qui cherche d'abord à attribuer une valeur puis à prendre une action gloutonne.

Exemple 2 : gridworld avec alias

Considérons un petit monde quadrillé dans lequel certaines cases distinctes ne peuvent pas être distinguées par la représentation utilisée par l'agent. Plus précisément, on suppose que les deux cases grises de la rangée du haut ont les mêmes descripteurs (*features*) du point de vue de l'algorithme. C'est une forme d'*aliasing perceptif* (*perceptual aliasing*) : deux états réels différents sont vus comme identiques par le modèle.

La situation est la suivante. Les cases du bas aux extrémités contiennent un danger fort (tête de mort), tandis que la case centrale du bas contient le but, ici représenté par un trésor, associé à une forte récompense positive. Intuitivement, pour atteindre le but rapidement, il faut parfois aller vers la gauche, parfois vers la droite, selon la case grise où l'on se trouve réellement. Or, si ces deux cases grises sont représentées exactement de la même manière, une méthode fondée sur les valeurs rencontrera une difficulté structurelle : elle sera poussée à attribuer la même valeur estimée aux mêmes actions dans ces deux situations pourtant différentes.



(a) Deux cases grises distinctes sont vues comme identiques par la représentation.

(a) Deux cases grises distinctes sont vues comme identiques par la représentation.

(c) En *policy based RL*, une politique stochastique peut sortir de l'impasse.

FIGURE 1.2 – Exemple de gridworld avec aliasing perceptif. Deux états réels distincts partagent la même représentation, ce qui peut rendre une politique déterministe inadaptée.

Pour rendre cela un peu plus explicite, imaginons que la politique ou la fonction de valeur d'action soit construite à partir d'un vecteur de caractéristiques $\phi(s, a)$. Si les deux cases grises partagent la même représentation, alors, du point de vue de l'algorithme, elles sont indiscernables. Une approximation de la forme

$$\hat{q}(s, a, \mathbf{w}) = f_{\mathbf{w}}(\phi(s, a))$$

attribuera donc les mêmes préférences d'action dans ces deux cases si $\phi(s, a)$ y est identique. Une politique gloutonne dérivée de $\hat{q}$ choisira alors la même direction dans les deux états.

C'est précisément ce qui crée ici un comportement d'oscillation. Si la politique déterministe choisit toujours *gauche* dans les deux cases grises, l'agent repart sans cesse vers la partie gauche du couloir. Si elle choisit toujours *droite*, il repart sans cesse vers la partie droite. Dans les deux cas, il peut rester coincé à alterner entre des positions voisines sans jamais descendre au bon endroit vers le trésor. Le problème n'est pas ici un simple manque d'entraînement : il vient du fait que la représentation ne permet pas d'exprimer la différence pertinente entre les deux situations.

Une politique stochastique permet au contraire une réponse plus souple. Au lieu d'imposer une action unique dans les deux cases grises, on peut apprendre une politique de la forme

$$\pi_{\theta}(a \mid s),$$

qui attribue une probabilité non nulle à plusieurs actions. Dans les cases aliasées, la politique peut par exemple donner une certaine probabilité à *gauche* et une certaine probabilité à *droite*. L'agent n'est alors plus condamné à reproduire indéfiniment la même action. Avec une probabilité positive, il finit par sortir de la boucle locale et par atteindre la case de récompense.

Cet exemple illustre bien une idée importante. Les méthodes fondées sur les valeurs cherchent souvent, au bout du compte, à construire une politique gloutonne ou presque gloutonne à partir d'une estimation de q . Lorsque la représentation d'état est imparfaite, ce passage par une règle déterministe peut être trop rigide. Les méthodes de gradient de la politique permettent au contraire d'apprendre directement une politique stochastique, ce qui peut être plus naturel et plus efficace lorsque plusieurs actions doivent rester possibles dans des situations que l'agent ne sait pas distinguer complètement.

Chapitre 2

Différences finies

2.1 Quel objectif $J(\theta)$ maximiser ?

Dans les méthodes de gradient de la politique, on se donne une famille de politiques paramétrées

$$\{\pi_\theta : \theta \in \mathbb{R}^d\}.$$

Avant même de parler de gradient, il faut préciser la fonction objectif que l'on cherche à maximiser.

L'idée générale est simple : pour chaque choix de paramètres θ , la politique π_θ induit un comportement, donc une performance moyenne. On note alors

$$J(\theta)$$

la performance associée à la politique π_θ . Les algorithmes de gradient de la politique cherchent à modifier θ de façon à augmenter cette quantité.

Le choix précis de J dépend du type de problème considéré.

Cas épisodique avec état initial fixé. Supposons qu'un épisode commence toujours dans un même état initial s_{init} . Une définition très naturelle est alors

$$J(\theta) = v_{\pi_\theta}(s_{\text{init}}),$$

où $v_{\pi_\theta}(s)$ désigne la valeur de l'état s sous la politique π_θ . Autrement dit, on mesure directement la qualité moyenne de la politique à partir du point de départ du problème.

Cas épisodique avec distribution initiale. Si l'état initial est aléatoire, de loi ρ_0 , on définit plus naturellement

$$J(\theta) = \sum_{s \in \mathcal{S}} \rho_0(s) v_{\pi_\theta}(s) = \mathbb{E}_{S_0 \sim \rho_0} [v_{\pi_\theta}(S_0)].$$

On ne cherche donc plus à maximiser la valeur d'un unique état initial, mais une moyenne pondérée sur les états de départ possibles.

Cas continuant. Dans un problème continuant, il n'y a pas de fin naturelle d'épisode, donc il n'est pas toujours commode de définir l'objectif à partir d'un état initial particulier. Une autre idée consiste alors à mesurer la performance moyenne de la politique lorsque le système fonctionne sur une longue durée.

On introduit pour cela une distribution d'occupation des états, notée $d_{\pi_\theta}(s)$, qui décrit la fréquence relative avec laquelle la politique π_θ visite les différents états. Suivant le cadre exact, il peut s'agir d'une distribution stationnaire ou d'une distribution moyenne d'occupation.

Un objectif classique est alors la *récompense moyenne par pas de temps* :

$$\eta(\theta) = \sum_{s \in \mathcal{S}} d_{\pi_\theta}(s) \sum_{a \in \mathcal{A}} \pi_\theta(a | s) \bar{r}(s, a),$$

où

$$\bar{r}(s, a) = \sum_{s' \in \mathcal{S}} P(s' | s, a) r(s, a, s')$$

désigne la récompense immédiate moyenne obtenue en choisissant l'action a dans l'état s .

Il faut bien comprendre le sens de l'expression « par pas de temps ». Elle ne signifie pas que l'objectif serait redéfini ou modifié à chaque itération de l'algorithme. Elle signifie que l'on regarde la récompense moyenne reçue à un instant typique, lorsque l'agent suit longtemps la politique π_θ . Autrement dit, on moyenne la récompense immédiate sur les états visités et sur les actions choisies par la politique.

Cette quantité $\bar{r}(s, a)$ ne doit pas être confondue avec une valeur d'action $q_{\pi_\theta}(s, a)$. En effet, $\bar{r}(s, a)$ ne prend en compte que la récompense immédiate moyenne au prochain pas, tandis que $q_{\pi_\theta}(s, a)$ représente un retour cumulé futur à partir du couple (s, a) .

Dans ce cadre, on note souvent l'objectif $\eta(\theta)$ plutôt que $J(\theta)$, précisément pour rappeler qu'il s'agit ici d'un critère de récompense moyenne de long terme, et non de la valeur d'un état initial ou d'un retour actualisé sur un épisode.

Ces objectifs sont proches dans leur esprit : ils mesurent tous la qualité moyenne de la politique paramétrée π_θ . Dans ce chapitre, pour fixer les idées, on raisonnera surtout dans le cadre épisodique. On pourra donc penser principalement à

$$J(\theta) = v_{\pi_\theta}(s_{\text{init}}) \quad \text{ou} \quad J(\theta) = \mathbb{E}_{S_0 \sim \rho_0} [v_{\pi_\theta}(S_0)].$$

2.2 Principe général : ascension de gradient

Une fois l'objectif $J(\theta)$ fixé, le problème devient un problème d'optimisation continue :

$$\max_{\theta \in \mathbb{R}^d} J(\theta).$$

Les méthodes de gradient de la politique cherchent en pratique non pas nécessairement un maximum global, mais un *maximum local* de cette fonction objectif.

Si J est dérivable par rapport aux paramètres, on peut considérer son gradient

$$\nabla_{\theta} J(\theta) = \begin{pmatrix} \frac{\partial J}{\partial \theta_1}(\theta) \\ \vdots \\ \frac{\partial J}{\partial \theta_d}(\theta) \end{pmatrix}.$$

Ce vecteur indique, au premier ordre, dans quelle direction il faut déplacer les paramètres pour augmenter le plus rapidement possible la valeur de J .

On obtient alors une mise à jour de type montée de gradient :

$$\theta_{k+1} = \theta_k + \alpha_k \nabla_{\theta} J(\theta_k),$$

où $\alpha_k > 0$ désigne le pas d'apprentissage (*learning rate*).

À ce stade, la difficulté n'est pas encore algorithmique mais analytique : comment calculer, ou au moins approximer, le gradient $\nabla_{\theta} J(\theta)$? La première idée, la plus directe, consiste à approcher ce gradient par différences finies.

2.3 Principe général des différences finies

Considérons d'abord une fonction quelconque

$$J : \mathbb{R}^d \rightarrow \mathbb{R},$$

supposée dérivable. Si l'on veut approximer sa dérivée partielle par rapport à la coordonnée θ_i , on peut utiliser la formule de différence finie centrée

$$\frac{\partial J}{\partial \theta_i}(\theta) \approx \frac{J(\theta + h\mathbf{e}_i) - J(\theta - h\mathbf{e}_i)}{2h},$$

où $h > 0$ est petit et où $\mathbf{e}_i$ désigne le i -ème vecteur de la base canonique de $\mathbb{R}^d$.

Cette approximation compare la valeur de la fonction en deux points voisins, obtenus en perturbant uniquement la i -ème composante de θ . On peut aussi utiliser une différence avant,

$$\frac{\partial J}{\partial \theta_i}(\theta) \approx \frac{J(\theta + h\mathbf{e}_i) - J(\theta)}{h},$$

mais la différence centrée est en général plus précise du point de vue numérique.

L'idée générale est donc la suivante : pour chaque dimension i , on perturbe légèrement le paramètre θ_i , on observe comment la performance varie, puis on en déduit une approximation de la dérivée correspondante. En répétant cette opération pour $i = 1, \dots, d$, on obtient un estimateur du gradient complet.

Cette approche est très simple conceptuellement :

- elle fonctionne quelle que soit la paramétrisation de la politique ;
- elle ne demande aucune formule analytique pour le gradient ;
- elle considère seulement $J(\theta)$ comme une fonction boîte noire que l'on sait évaluer.

2.4 Application à une politique paramétrée

Appliquons maintenant cette idée au cas du RL. Dans le cadre épisodique, une politique π_{θ} induit une distribution sur les trajectoires, que l'on notera $p_{\theta}(\tau)$. Si

$$\tau = (S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T)$$

désigne une trajectoire complète, on peut écrire l'objectif sous la forme

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}} [R(\tau)],$$

où $R(\tau)$ désigne le retour total associé à la trajectoire τ .

Comme dans les chapitres précédents, cette espérance n'est en général pas calculable exactement. On l'approche donc par une moyenne empirique sur un batch de m trajectoires indépendantes :

$$\hat{J}_m(\theta) = \frac{1}{m} \sum_{k=1}^m R(\tau^{(k)}), \quad \tau^{(k)} \sim p_{\theta}.$$

On peut alors approximer la i -ème dérivée partielle par

$$\widehat{\frac{\partial J}{\partial \theta_i}}(\theta) = \frac{\hat{J}_m(\theta + h\mathbf{e}_i) - \hat{J}_m(\theta - h\mathbf{e}_i)}{2h}.$$

En regroupant ces dérivées coordonnées par coordonnées, on obtient un estimateur du gradient, puis une mise à jour d'ascension de gradient.

Sur le papier, la méthode est séduisante. Elle ne nécessite ni équation de Bellman, ni dérivation de la loi des trajectoires, ni formule spéciale propre au RL. Il suffit, en apparence, d'essayer des paramètres voisins, d'estimer leur performance moyenne, puis de comparer.

2.5 Exemple : AIBO

Un exemple classique d'apprentissage par différences finies sur un système robotique réel est donné par le travail de Kohl et Stone sur le robot quadrupède Sony AIBO [23]. L'objectif est d'apprendre une marche avant aussi rapide que possible pour un robot à quatre pattes, dans un contexte motivé notamment par RoboCup.

L'idée générale du papier s'accorde très bien avec le point de vue de ce chapitre. On se donne d'abord une politique paramétrée, puis on cherche à améliorer ses paramètres à partir de perturbations locales de cette politique, sans utiliser d'expression analytique exacte du gradient.

Politique paramétrée. Dans cet exemple, la politique n'est pas une politique état-action au sens classique d'un petit MDP tabulaire. Il s'agit plutôt d'une commande paramétrée de la marche du robot. Les auteurs représentent la locomotion par un ensemble de paramètres décrivant la trajectoire des pieds, la posture du corps et certaines constantes temporelles du cycle de marche.

Le problème devient ainsi une optimisation dans un espace continu de paramètres. Plus précisément, l'allure (*gait*) retenue dans l'article est décrite par un vecteur

$$\theta \in \mathbb{R}^{12}.$$

Chaque valeur de θ définit une manière particulière de faire marcher le robot, donc une politique au sens large : une règle de commande complète, ici essentiellement en boucle ouverte, à l'exception de petites corrections destinées à compenser le bruit. Autrement dit, au lieu d'apprendre une table de valeurs ou une fonction $q(s, a)$, on apprend directement les paramètres d'une commande de locomotion.

Objectif à maximiser. L'objectif choisi par les auteurs est volontairement simple : maximiser la vitesse de déplacement vers l'avant. On peut donc voir la fonction objectif comme

$$J(\theta) = \text{vitesse moyenne vers l'avant obtenue avec la marche définie par } \theta.$$

Tant que le robot ne tombe pas, le critère principal n'est pas ici une notion abstraite de stabilité, mais bien la rapidité du déplacement.

Ce choix est intéressant pédagogiquement, car il montre qu'en pratique une méthode de type policy gradient peut être utilisée même lorsque la récompense n'est pas donnée sous la forme d'un petit signal scalaire à chaque instant. Ici, la performance est essentiellement évaluée à partir d'une mesure globale du comportement produit par la politique.

Estimation du gradient par perturbations. Comme la dépendance exacte de la performance par rapport aux paramètres est inconnue, les auteurs ne calculent pas $\nabla_{\theta} J(\theta)$ analytiquement. Ils l'approximent par une méthode fondée sur des perturbations aléatoires autour de la politique courante.

On part d'un vecteur de paramètres courant

$$\theta = (\theta_1, \dots, \theta_N), \quad N = 12.$$

À chaque itération, on génère plusieurs politiques voisines

$$\theta^{(i)} = (\theta_1 + \Delta_1^{(i)}, \dots, \theta_N + \Delta_N^{(i)}),$$

où chaque perturbation $\Delta_j^{(i)}$ est choisie dans l'ensemble $\{-\varepsilon_j, 0, +\varepsilon_j\}$, les quantités $\varepsilon_j > 0$ étant petites devant les paramètres correspondants.

Pour chaque dimension n , on regroupe ensuite les politiques testées en trois ensembles $S_{+\varepsilon,n}$, $S_{0,n}$, $S_{-\varepsilon,n}$, suivant que la n -ième composante a été perturbée positivement, laissée inchangée ou perturbée négativement. En moyennant les scores observés dans chacun de ces trois groupes, on obtient trois quantités $\text{Avg}_{+\varepsilon,n}$, $\text{Avg}_{0,n}$, $\text{Avg}_{-\varepsilon,n}$, qui donnent une indication sur l'effet d'une petite variation de θ_n .

L'ajustement dans la direction n est alors construit de manière heuristique :

$$A_n = \begin{cases} 0, & \text{si } \text{Avg}_{0,n} > \text{Avg}_{+\varepsilon,n} \text{ et } \text{Avg}_{0,n} > \text{Avg}_{-\varepsilon,n}, \\ \text{Avg}_{+\varepsilon,n} - \text{Avg}_{-\varepsilon,n}, & \text{sinon.} \end{cases}$$

Une fois le vecteur $A = (A_1, \dots, A_N)$ obtenu, il est normalisé puis multiplié par un pas $\eta > 0$, ce qui donne une mise à jour de la forme

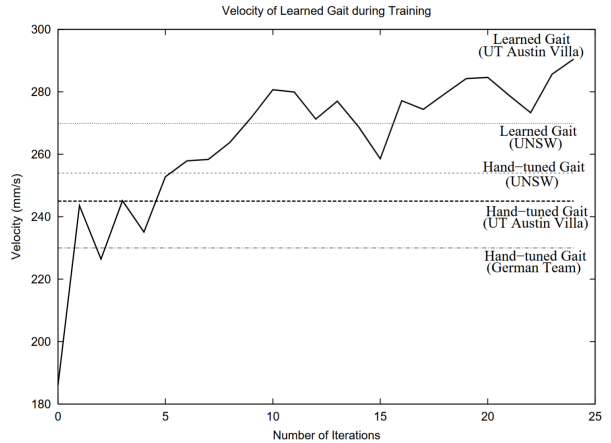
$$\theta \leftarrow \theta + \eta \frac{A}{\|A\|}.$$

Ce n'est pas exactement la différence finie centrée la plus élémentaire étudiée plus haut, mais l'esprit est le même : on perturbe localement les paramètres, on observe l'effet de ces perturbations sur la performance, puis on ajuste la politique dans une direction jugée favorable.

Évaluation sur robots réels. L'un des aspects les plus intéressants de cet exemple est que l'apprentissage n'est pas effectué dans un simulateur, mais directement sur des robots physiques. Les AIBO évoluent entre des repères fixes sur le terrain et mesurent le temps nécessaire pour parcourir la distance imposée. Une politique est d'autant meilleure que le robot traverse plus vite la zone d'évaluation.



(a) Environnement d'entraînement.



(b) Amélioration de la vitesse au cours de l'entraînement.

FIGURE 2.1 – Exemple d'apprentissage par perturbations de politique sur le robot AIBO [23]. (a) Environnement d'entraînement utilisé pour l'évaluation des politiques sur les robots AIBO. Chaque robot se déplace entre une paire de repères fixes et mesure son temps de parcours. (b) Performance du meilleur gait trouvé au cours de l'entraînement. On observe une amélioration globale, avec des oscillations liées au bruit expérimental et aux choix d'hyperparamètres.

Cette procédure rend l'exemple particulièrement concret. Évaluer $J(\theta)$ signifie ici : charger les paramètres sur le robot, le faire marcher, mesurer sa performance, puis recommencer avec d'autres paramètres.

Dans l'implémentation décrite par les auteurs, chaque itération utilise $t = 15$ politiques perturbées. Comme les mesures sont bruitées, chaque politique est évaluée trois fois, puis les scores sont moyennés. Une itération représente donc un coût expérimental non négligeable.

Résultats observés. Les résultats rapportés dans l'article montrent une amélioration nette de la vitesse au cours de l'entraînement. La courbe de performance du meilleur gait trouvé à chaque itération présente une tendance globale à la hausse, malgré des fluctuations dues au bruit des évaluations.

Après une vingtaine d'itérations, les auteurs obtiennent une marche significativement plus rapide que leur point de départ, et même supérieure à plusieurs gait conçus à la main. Le résultat final illustre donc bien que des méthodes d'optimisation directe de politique peuvent être efficaces sur un système réel, même lorsque l'on ne dispose pas d'une formule analytique simple pour le gradient.

Pourquoi cet exemple est-il instructif ? Cet exemple montre très concrètement ce que signifie « apprendre par différences finies » dans un problème réel :

- la politique est représentée par un petit nombre de paramètres continus ;
- la performance est observée expérimentalement, et non calculée à partir d'un modèle exact ;
- l'amélioration se fait par comparaison entre politiques voisines ;
- chaque estimation du gradient coûte cher, car elle nécessite plusieurs essais physiques.

Il montre aussi les deux faces de l'approche. D'un côté, la méthode est simple, générale et directement applicable à un robot réel. De l'autre, elle reste coûteuse en évaluations, sensible au bruit et délicate à régler. C'est précisément pour ces raisons que, dans les problèmes de plus grande dimension, on préfère en général des estimateurs de gradient qui exploitent davantage la structure probabiliste de la politique elle-même.

2.6 Pourquoi cette approche est-elle peu satisfaisante en RL ?

Malgré sa simplicité, l'approche par différences finies est rarement utilisée telle quelle dans les algorithmes classiques de gradient de la politique. Plusieurs raisons se cumulent.

Coût en grande dimension. Pour approximer le gradient complet par différences finies centrées, il faut en principe deux évaluations de performance par coordonnée, donc environ $2d$ estimations de J . Si la politique contient beaucoup de paramètres, ce coût devient rapidement prohibitif.

Bruit statistique. Chaque estimation $\hat{J}_m(\theta)$ est bruitée, puisqu'elle repose sur un nombre fini de trajectoires. Or l'estimation d'une dérivée par différence finie consiste à soustraire deux quantités bruitées puis à diviser par un petit nombre h . Cela peut produire une très forte variance. Si h est trop grand, l'approximation de dérivée est grossière ; s'il est trop petit, le bruit domine.

Absence d'exploitation de la structure séquentielle. La méthode traite la politique comme une boîte noire et n'utilise qu'un seul nombre par trajectoire : sa performance finale. Pourtant, une trajectoire contient beaucoup plus d'information. Elle révèle quelles actions ont été choisies, avec quelles probabilités, et comment les récompenses se sont accumulées dans le temps. Une bonne méthode de RL devrait exploiter cette structure interne plutôt que de se limiter à comparer des performances globales.

Peu d'efficacité en données. Les trajectoires générées pour estimer $J(\theta + he_i)$ ne servent pas directement à estimer $J(\theta + he_j)$ lorsque $i \neq j$. L'information recueillie est donc peu mutualisée entre les différentes composantes du paramètre.

On peut résumer ainsi la situation : les différences finies fournissent une méthode générale, simple et conceptuellement utile pour comprendre ce qu'est un gradient, mais elles restent trop coûteuses et trop bruitées pour constituer la base des méthodes modernes de gradient de la politique.

C'est précisément ce qui motive le chapitre suivant. Au lieu d'approximer le gradient *de l'extérieur*, en perturbant les paramètres, on cherchera une formule *interne* qui exprime $\nabla_{\theta} J(\theta)$ comme une espérance sous la politique courante. C'est cette idée qui conduit à l'astuce de la fonction de score et au théorème du gradient de la politique.

Chapitre 3

Gradient de la politique

3.1 Fonction de score

L'astuce *likelihood ratio gradient* ou *policy gradient*

La pierre de base de tout le chapitre est une identité très générale, connue sous les noms de *likelihood ratio trick*, *score function estimator* ou, dans le contexte RL, *policy gradient trick*.

Considérons une variable aléatoire X de loi ou de densité p_θ dépendant d'un vecteur de paramètres θ , ainsi qu'une fonction mesurable f . On veut calculer le gradient de

$$J(\theta) = \mathbb{E}_{X \sim p_\theta} [f(X)].$$

Sous des hypothèses régulières permettant d'échanger dérivation et intégration, on obtient

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int f(x) p_\theta(x) dx \\ &= \int f(x) \nabla_\theta p_\theta(x) dx. \end{aligned}$$

Or l'identité élémentaire

$$\nabla_\theta p_\theta(x) = p_\theta(x) \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} = p_\theta(x) \nabla_\theta \log p_\theta(x)$$

permet d'écrire

$$\nabla_\theta J(\theta) = \int f(x) p_\theta(x) \nabla_\theta \log p_\theta(x) dx = \mathbb{E}_{X \sim p_\theta} [f(X) \nabla_\theta \log p_\theta(X)].$$

Le vecteur

$$\nabla_\theta \log p_\theta(x)$$

est appelé *fonction de score* (*score function*). Il mesure, en quelque sorte, la sensibilité locale du logarithme de la probabilité au paramètre.

Cette identité est très précieuse, car elle transforme un gradient d'espérance en une espérance d'une quantité calculable à partir d'échantillons de la loi actuelle. C'est exactement le genre de formule dont on a besoin dans les méthodes RL de type *policy gradient*.

Exemple de score : softmax

Supposons que l'espace d'actions soit discret. Une manière naturelle de paramétrer une politique est d'associer à chaque couple (s, a) un *score linéaire* par rapport au description de l'état $x(s, a)^\top$

$$h_\theta(s, a) = x(s, a)^\top \theta,$$

puis de transformer ces scores en probabilités par une loi *softmax* :

$$\pi_\theta(a | s) = \frac{\exp(x(s, a)^\top \theta)}{\sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta)}.$$

Les quantités $h_\theta(s, a)$ jouent ici le rôle de *scores* ou de *préférences* pour les actions. Elles sont conceptuellement proches des valeurs d'action $q(s, a)$: plus $h_\theta(s, a)$ est grand, plus l'action a est jugée favorable dans l'état s . Dans une approche gloutonne, on choisirait directement l'action qui maximise ce score. La softmax constitue une autre manière, plus régulière, de transformer ces scores en politique : les actions les mieux notées sont favorisées, mais toutes gardent en général une probabilité strictement positive, ce qui conduit à une politique stochastique différentiable bien adaptée aux méthodes de gradient. Cette paramétrisation a une propriété très utile : on peut calculer explicitement la *fonction de score*

$$\nabla_\theta \log \pi_\theta(a | s),$$

qui apparaît dans les méthodes de gradient de politique.

Détaillons ce calcul (partie optionnelle). Partons de l'expression de la politique et prenons le logarithme :

$$\log \pi_\theta(a | s) = \log \left(\frac{\exp(x(s, a)^\top \theta)}{\sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta)} \right).$$

En utilisant $\log(A/B) = \log A - \log B$, on obtient

$$\log \pi_\theta(a | s) = \log(\exp(x(s, a)^\top \theta)) - \log\left(\sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta)\right),$$

donc

$$\log \pi_\theta(a | s) = x(s, a)^\top \theta - \log\left(\sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta)\right).$$

On dérive maintenant terme à terme par rapport à θ .

Pour le premier terme,

$$\nabla_\theta(x(s, a)^\top \theta) = x(s, a).$$

Pour le second on pose

$$Z_\theta(s) = \sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta),$$

et on utilise la règle

$$\nabla_\theta \log Z_\theta(s) = \frac{1}{Z_\theta(s)} \nabla_\theta Z_\theta(s).$$

Or

$$\nabla_\theta Z_\theta(s) = \nabla_\theta \sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta) = \sum_{b \in \mathcal{A}} \nabla_\theta \exp(x(s, b)^\top \theta).$$

Comme $\nabla_\theta e^{f(\theta)} = e^{f(\theta)} \nabla_\theta f(\theta)$, on obtient

$$\nabla_\theta Z_\theta(s) = \sum_{b \in \mathcal{A}} \exp(x(s, b)^\top \theta) x(s, b).$$

Ainsi,

$$\nabla_{\theta} \log Z_{\theta}(s) = \frac{1}{\sum_{b \in \mathcal{A}} \exp(x(s, b)^{\top} \theta)} \sum_{b \in \mathcal{A}} \exp(x(s, b)^{\top} \theta) x(s, b).$$

Donc

$$\nabla_{\theta} \log Z_{\theta}(s) = \sum_{b \in \mathcal{A}} \frac{\exp(x(s, b)^{\top} \theta)}{\sum_{b \in \mathcal{A}} \exp(x(s, b)^{\top} \theta)} x(s, b).$$

On reconnaît alors les probabilités softmax $\pi_{\theta}(b | s)$ et en réunissant les deux morceaux, on trouve

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = x(s, a) - \sum_{b \in \mathcal{A}} \pi_{\theta}(b | s) x(s, b).$$

Autrement dit,

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = x(s, a) - \mathbb{E}_{B \sim \pi_{\theta}(\cdot | s)} [x(s, B)].$$

Cette formule est très instructive. Le gradient du log de la probabilité d'une action compare les caractéristiques de l'action effectivement choisie à la moyenne des caractéristiques des actions possibles sous la politique courante. Ainsi, augmenter $\log \pi_{\theta}(a | s)$ revient à rendre l'action a relativement plus attractive que les autres, et pas simplement à augmenter tous les scores en même temps.

Exemple de score : politique gaussienne

Pour des actions continues, une politique gaussienne est un choix très fréquent. Dans le cas scalaire, on peut écrire

$$(A_t | S_t = s) \sim \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}^2(s)).$$

La densité associée vaut

$$\pi_{\theta}(a | s) = \frac{1}{\sqrt{2\pi} \sigma_{\theta}(s)} \exp\left(-\frac{(a - \mu_{\theta}(s))^2}{2\sigma_{\theta}^2(s)}\right).$$

En prenant le logarithme, on obtient

$$\log \pi_{\theta}(a | s) = -\log \sigma_{\theta}(s) - \frac{1}{2} \log(2\pi) - \frac{(a - \mu_{\theta}(s))^2}{2\sigma_{\theta}^2(s)}.$$

Si l'on suppose d'abord que seule la moyenne dépend des paramètres et que la variance est fixée, alors

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = \frac{a - \mu_{\theta}(s)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(s).$$

Pour une paramétrisation linéaire $\mu(s) = x(s)^{\top} \theta$ on obtient

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = \frac{a - \mu_{\theta}(s)}{\sigma^2} x(s).$$

Cette formule a une interprétation intuitive. Si l'action réalisée a est supérieure à la moyenne courante $\mu_{\theta}(s)$, alors le coefficient multiplicatif est positif; si elle est inférieure, il est négatif. Le gradient pousse donc la moyenne dans la direction des actions associées à de bons retours.

Si la variance est elle aussi paramétrée, on obtient une formule un peu plus longue. En pratique, on paramètre souvent $\log \sigma_{\theta}(s)$ plutôt que $\sigma_{\theta}(s)$ lui-même, afin de garantir la positivité et d'améliorer la stabilité numérique.

3.2 Gradient de la politique

Fonction objectif

Pour fixer les idées, plaçons-nous d'abord dans un cadre épisodique, avec un état initial fixé s . La politique π_θ induit alors une distribution sur les trajectoires complètes partant de cet état. On notera une trajectoire réalisée

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_{T-1}, a_{T-1}, r_T, s_T), \quad s_0 = s,$$

où s_T est un état terminal.

À cette trajectoire, on associe le retour total

$$R(\tau) = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots + \gamma^{T-1} r_T = \sum_{t=0}^{T-1} \gamma^t r_{t+1}.$$

Dans le cas particulier $\gamma = 1$, on retrouve simplement la somme des récompenses sur l'épisode fini.

Notons maintenant

$$p_\theta(\tau | s)$$

la probabilité de la trajectoire τ lorsque l'on suit la politique π_θ à partir de l'état initial s . Il s'agit de la distribution de trajectoires induite conjointement par la politique et par l'environnement.

La quantité naturelle à maximiser est alors la performance moyenne de la politique à partir de s , c'est-à-dire

$$J(\theta) = v_{\pi_\theta}(s).$$

Comme $v_{\pi_\theta}(s)$ est l'espérance du retour sous la politique π_θ , on peut aussi écrire

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\cdot | s)} [R(\tau)] = \sum_{\tau} p_\theta(\tau | s) R(\tau). \quad (3.1)$$

Autrement dit, optimiser la politique revient à choisir les paramètres θ de façon à augmenter la moyenne des retours des trajectoires qu'elle produit.

Si l'état initial n'est pas fixé mais tiré selon une distribution initiale ρ , on remplace simplement $v_{\pi_\theta}(s)$ par une moyenne sur les états initiaux :

$$J(\theta) = \mathbb{E}_{s_0 \sim \rho} [v_{\pi_\theta}(s_0)] = \sum_{s_0 \in \mathcal{S}} \rho(s_0) v_{\pi_\theta}(s_0).$$

On peut alors aussi écrire

$$J(\theta) = \sum_{s_0 \in \mathcal{S}} \rho(s_0) \sum_{\tau} p_\theta(\tau | s_0) R(\tau).$$

Pour ne pas alourdir les notations, on se place désormais dans le cas où l'état initial est fixé, $s_0 = s$, et où l'objectif est donné par l'équation (3.1). Dans le cas où l'état initial est distribué selon une loi initiale, la dérivation est de même nature.

Application de l'astuce pour calculer le gradient

Nous voulons maintenant calculer le gradient de la fonction objectif. En repartant de l'expression précédente,

$$J(\theta) = \sum_{\tau} p_\theta(\tau | s) R(\tau),$$

on obtient, sous des hypothèses régulières permettant d'échanger dérivation et somme,

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \sum_{\tau} p_{\theta}(\tau | s) R(\tau) = \sum_{\tau} \nabla_{\theta} p_{\theta}(\tau | s) R(\tau).$$

On utilise alors l'identité vue à la section précédente,

$$\nabla_{\theta} f(\theta) = f(\theta) \nabla_{\theta} \log f(\theta),$$

appliquée ici à $f(\theta) = p_{\theta}(\tau | s)$. On obtient

$$\nabla_{\theta} p_{\theta}(\tau | s) = p_{\theta}(\tau | s) \nabla_{\theta} \log p_{\theta}(\tau | s),$$

puis

$$\nabla_{\theta} J(\theta) = \sum_{\tau} p_{\theta}(\tau | s) \nabla_{\theta} \log p_{\theta}(\tau | s) R(\tau).$$

C'est donc une espérance sous la politique courante :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\cdot | s)} \left[R(\tau) \nabla_{\theta} \log p_{\theta}(\tau | s) \right].$$

À ce stade, le résultat est déjà important : le gradient de l'objectif s'exprime comme une espérance d'une quantité calculée sur les trajectoires engendrées par la politique elle-même.

Décomposition de la probabilité d'une trajectoire

Pour rendre cette formule exploitable, il faut maintenant comprendre la structure de

$$p_{\theta}(\tau | s).$$

Dans un MDP standard, si l'état initial est fixé, la probabilité d'une trajectoire se factorise sous la forme

$$p_{\theta}(\tau | s) = \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) P(s_{t+1} | s_t, a_t).$$

La politique intervient donc à travers les probabilités des actions choisies, tandis que la dynamique de l'environnement intervient à travers les probabilités de transition.

En prenant le logarithme, le produit devient une somme :

$$\log p_{\theta}(\tau | s) = \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) + \sum_{t=0}^{T-1} \log P(s_{t+1} | s_t, a_t).$$

En dérivant par rapport aux paramètres θ , les termes de dynamique disparaissent, car l'environnement n'est pas paramétré par θ . Il reste donc

$$\nabla_{\theta} \log p_{\theta}(\tau | s) = \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

En reportant cela dans l'expression du gradient, on obtient

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\cdot | s)} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right].$$

C'est la forme la plus simple de la formule du gradient de la politique.

Il faut bien remarquer ce qui s'est passé. Le calcul du gradient n'a pas consisté à différencier explicitement la dynamique de l'environnement. Toute la dépendance en θ passe par la politique, donc par les termes $\log \pi_{\theta}(a_t | s_t)$.

Espérance approchée par une moyenne sur un batch de m trajectoires

Comme dans les chapitres précédents, l'espérance théorique n'est pas calculée exactement. On la remplace par une moyenne empirique à partir d'un batch de trajectoires

$$\tau^{(1)}, \dots, \tau^{(m)} \sim p_{\theta}(\cdot | s).$$

Si la i -ème trajectoire a longueur T_i , on obtient l'estimateur

$$\widehat{\nabla_{\theta} J}(\theta) = \frac{1}{m} \sum_{i=1}^m R(\tau^{(i)}) \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}).$$

Une mise à jour naturelle des paramètres est alors

$$\theta \leftarrow \theta + \alpha \widehat{\nabla_{\theta} J}(\theta),$$

où $\alpha > 0$ est le pas d'apprentissage.

Cette formule possède une interprétation intuitive. Si une trajectoire obtient un grand retour, on augmente la probabilité des actions qui ont été choisies le long de cette trajectoire. Si, au contraire, le retour est faible, la correction associée pousse à réduire relativement leur probabilité. Bien sûr, cette intuition reste globale et encore un peu grossière : on utilise ici le même retour total $R(\tau)$ pour tous les instants de la trajectoire, ce qui conduit à une variance importante. Ce point sera précisément discuté dans la suite.

Considérations pratiques. Pour appliquer cette procédure, on n'a pas besoin de calculer à la main le gradient de toute la politique considérée comme une fonction compliquée. Ce qu'il faut savoir calculer, pour chaque transition observée, c'est la quantité $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$. Lorsque la politique appartient à une famille simple, comme la loi softmax ou la loi gaussienne, on peut écrire ce gradient analytiquement, comme on l'a vu plus haut.

Lorsque la politique est représentée par un réseau de neurones, le principe reste le même. Dans l'implémentation réelle, on n'a pas besoin d'une « cible » au sens de l'apprentissage supervisé pour utiliser la rétropropagation. La différentiation automatique ne demande pas une cible explicite ; elle demande seulement une quantité scalaire différentiable par rapport aux paramètres. Ici, à partir d'un batch de trajectoires déjà collectées, on construit précisément une telle quantité, par exemple

$$L_{PG}(\theta) = -\frac{1}{m} \sum_{i=1}^m R(\tau^{(i)}) \sum_{t=0}^{T_i-1} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}).$$

Le signe moins est choisi pour pouvoir utiliser une routine standard de descente de gradient. Minimiser $L_{PG}(\theta)$ revient alors à maximiser l'estimateur précédent de $\nabla_{\theta} J(\theta)$.

Il faut bien comprendre ce qui dépend encore de θ dans cette expression. Une fois les trajectoires échantillonnées, les états $s_t^{(i)}$, les actions $a_t^{(i)}$ et les retours $R(\tau^{(i)})$ sont traités comme des données fixées. La seule dépendance en θ reste dans les termes $\log \pi_{\theta}(a_t^{(i)} | s_t^{(i)})$. La rétropropagation calcule donc simplement le gradient de cette fonction scalaire par rapport aux paramètres de la politique. Autrement dit, on ne rétropropage pas *à travers l'environnement* ; on rétropropage seulement à travers la politique paramétrée.

Indépendance au modèle

Un des attraits majeurs de cette méthode est son indépendance explicite au modèle. En effet, pour calculer la contribution d'une trajectoire observée au gradient, il suffit de connaître

$$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

aux différents instants. Il n'est pas nécessaire de connaître la loi de transition

$$P(s' | s, a).$$

Cela ne signifie pas que l'environnement ne joue aucun rôle. Au contraire, c'est lui qui détermine les états visités et les récompenses reçues, donc les trajectoires effectivement observées. Mais il n'apparaît pas dans la partie différenciée de la formule. C'est en ce sens précis que la méthode est *model-free* : on peut estimer le gradient à partir d'échantillons d'interaction, sans disposer d'un modèle explicite de la dynamique.

En résumé, la méthode repose sur trois idées simples :

1. Écrire l'objectif comme une espérance de retour sur les trajectoires ;
2. Utiliser l'identité de la fonction de score pour faire apparaître $\nabla_{\theta} \log p_{\theta}(\tau | s)$;
3. Exploiter la factorisation d'une trajectoire pour ramener ce terme à une somme de gradients de log-probabilités d'actions.

On dispose ainsi d'une première formule générale du gradient de la politique. Elle est fondamentale, mais encore imparfaite sur le plan pratique : l'estimateur obtenu est souvent très bruité. Les méthodes de Monte Carlo policy gradient et d'actor-critic chercheront précisément à conserver cette idée de base tout en améliorant l'efficacité.

3.3 Monte Carlo Policy Gradient

Structure temporelle

La formule obtenue à la section précédente s'écrit, dans le cadre épisodique,

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right].$$

Cette expression est correcte, mais elle cache une structure temporelle importante. En effet, le même retour total $R(\tau)$ intervient pour tous les instants t , alors qu'une action choisie au temps t ne peut pas influencer les récompenses déjà reçues avant cet instant.

Pour faire apparaître cette idée, écrivons le retour total comme une somme des récompenses futures :

$$R(\tau) = \sum_{t'=0}^{T-1} \gamma^{t'} R_{t'+1}.$$

En le remplaçant dans la formule précédente, on obtient

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t'=0}^{T-1} \gamma^{t'} R_{t'+1} \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right].$$

En développant cette double somme, on rencontre des termes de la forme

$$\gamma^{t'} R_{t'+1} \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

Or, lorsque $t' \geq t$, la récompense $R_{t'+1}$ appartient au futur de la décision prise au temps t , donc il est plausible qu'elle contribue au signal d'apprentissage associé à cette décision. En revanche, lorsque $t' < t$, la récompense $R_{t'+1}$ appartient au passé par rapport à l'action A_t , et elle ne doit pas influencer la correction apportée à cette action.

Le résultat standard — pour une démonstration voir [24] — est que les termes correspondant au passé s'annulent en espérance. Le point clé derrière cette annulation est l'identité

$$\mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(A_t | S_t) | S_t] = 0,$$

qui a déjà été mentionnée implicitement dans la discussion sur la fonction de score. Après simplification, il ne reste donc que la partie future du retour à partir de l'instant t .

On introduit alors le retour à partir du temps t ,

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T = \sum_{t'=t}^{T-1} \gamma^{t'-t} R_{t'+1}.$$

La formule du gradient devient

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{T-1} G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right].$$

Cette écriture est plus fine que la précédente : à chaque instant t , on pondère le gradient de la log-probabilité de l'action choisie non pas par tout le retour de l'épisode, mais seulement par la partie du retour qui lui est postérieure. Autrement dit, la structure temporelle du problème est maintenant visible : la décision prise au temps t est reliée à ce qui lui arrive ensuite, c'est-à-dire au retour futur G_t .

Pourquoi ce raffinement est-il important ? L'écriture avec le retour total $R(\tau)$ est déjà correcte : elle fournit un estimateur non biaisé du gradient. Mais elle est souvent peu efficace en pratique. En effet, elle associe à l'action prise au temps t des récompenses situées avant cet instant, donc des termes aléatoires qui fluctuent d'un épisode à l'autre sans contenir d'information utile sur l'effet futur de cette action.

Le passage de $R(\tau)$ à G_t ne change pas la valeur moyenne du gradient estimé : l'estimateur reste non biaisé. En revanche, il réduit sa variance, parfois de manière très importante, car il élimine des contributions dont l'espérance est nulle mais dont les fluctuations ajoutent du bruit. C'est l'idée de *causalité* : le futur peut dépendre du présent, mais le passé ne dépend pas de l'action choisie maintenant.

Cette amélioration est essentielle pour rendre les méthodes Monte Carlo de gradient de politique réellement utilisables. Sans elle, l'estimateur fondé sur le retour total est souvent beaucoup trop bruité ; avec cette procédure, appelée *reward-to-go*, on obtient déjà une méthode nettement plus stable, comme dans REINFORCE.

Monte Carlo Policy Gradient (REINFORCE)

La méthode Monte Carlo la plus classique fondée sur cette idée est l'algorithme REINFORCE, introduit par Williams [15]. Il s'agit d'une méthode épisodique : on génère un épisode complet sous la politique courante, puis on utilise les retours Monte Carlo G_t pour corriger les paramètres de la politique.

La mise à jour élémentaire associée à l'instant t est

$$\theta \leftarrow \theta + \alpha G_t \nabla_{\theta} \log \pi_{\theta}(A_t | S_t),$$

où $\alpha > 0$ est le pas d'apprentissage.

L'interprétation est directe. Si le retour futur G_t est grand, on augmente la probabilité de l'action qui a été effectivement choisie au temps t . Si G_t est faible, la correction pousse au contraire à diminuer relativement cette probabilité.

Algorithme : REINFORCE (version conceptuelle)

1. initialiser les paramètres θ de la politique ;

2. générer un épisode complet

$$(S_0, A_0, R_1), (S_1, A_1, R_2), \dots, (S_{T-1}, A_{T-1}, R_T)$$

en suivant π_θ ;

3. pour chaque instant t , calculer le retour Monte Carlo

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T;$$

4. mettre à jour θ avec

$$\theta \leftarrow \theta + \alpha G_t \nabla_\theta \log \pi_\theta(A_t | S_t);$$

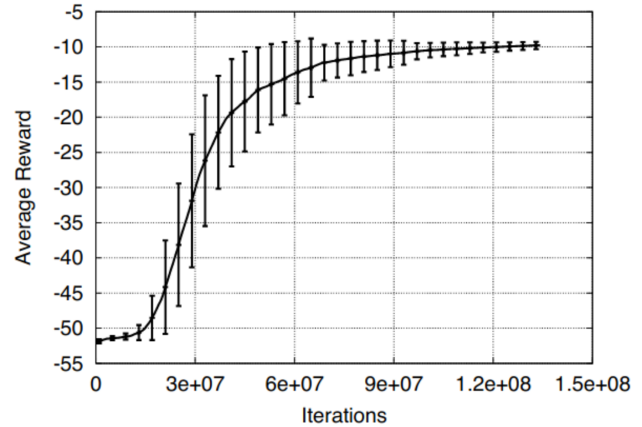
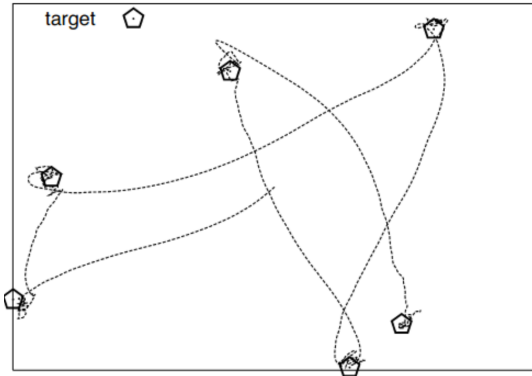
5. recommencer avec de nouveaux épisodes.

Du point de vue conceptuel, REINFORCE joue pour la politique un rôle analogue à celui des méthodes Monte Carlo vues dans RL2 pour les fonctions de valeur [2, section 2.1]. Dans les deux cas, on attend la fin de l'épisode pour construire un retour complet. La différence est que l'on ne met plus à jour une estimation de valeur, mais directement les paramètres de la politique.

Exemple : Puck World

Un exemple pédagogique classique est l'environnement *Puck World*. L'agent y contrôle un palet mobile dans le plan. L'état contient typiquement la position et la vitesse du palet, ainsi que la position d'une cible. L'objectif est de déplacer le palet de façon à se rapprocher de cette cible ; lorsque celle-ci est atteinte, ou après un certain temps, la cible est replacée ailleurs.

Cet exemple est particulièrement intéressant pour les méthodes de gradient de politique, car il met en jeu un contrôle naturellement continu. L'action peut être modélisée comme une petite force appliquée au palet, par exemple selon deux coordonnées. Dans un cadre purement value-based, un tel espace d'actions continues est moins commode à traiter directement. En revanche, une politique paramétrée peut produire une loi sur ces commandes, par exemple une loi gaussienne bidimensionnelle, ce qui rend le cadre policy gradient plus naturel.



(a) Environnement *Puck World* : le palet se déplace dans le plan sous l'effet de petites forces, et doit rejoindre des cibles successives.

(b) Récompense moyenne au cours de l'entraînement : la performance s'améliore, mais la convergence reste lente et bruitée.

FIGURE 3.1 – Illustration de *Puck World* et de l'apprentissage par une variante de Monte Carlo policy gradient.

La figure 3.1(a) donne une idée qualitative du problème. On y voit plusieurs trajectoires du palet se dirigeant vers des cibles successives. L'agent reçoit une récompense lorsqu'il se rapproche de la cible, et

la position de celle-ci est réinitialisée périodiquement. Le problème est donc dynamique : il ne s'agit pas seulement d'atteindre un point fixe, mais de s'adapter continuellement à une cible qui change.

Dans un tel contexte, REINFORCE s'interprète simplement :

- on exécute la politique sur un épisode complet ;
- on observe à quels instants les forces choisies ont rapproché ou éloigné le palet de la cible ;
- on calcule les retours G_t ;
- on augmente la probabilité des commandes associées à des retours futurs élevés.

La figure 3.1(b) montre que cette idée fonctionne effectivement : la récompense moyenne augmente au cours de l'entraînement, ce qui signifie que le palet apprend progressivement à mieux poursuivre sa cible. Mais cette figure met aussi en évidence une difficulté importante. L'amélioration est lente, et les barres d'erreur restent longtemps visibles. Autrement dit, l'apprentissage converge, mais au prix d'un très grand nombre d'itérations et avec une variance importante.

Cet exemple illustre donc bien les deux faces des méthodes Monte Carlo de gradient de politique. D'un côté, elles permettent d'apprendre directement une politique stochastique dans un problème à actions continues, sans construire explicitement une fonction $q(s, a)$ sur tout l'espace des actions. De l'autre, leur coût en données peut être considérable : la méthode progresse, mais elle le fait lentement, ce qui annonce naturellement la nécessité de techniques de réduction de variance plus efficaces.

Avantages et limites

Les méthodes Monte Carlo de gradient de politique ont plusieurs qualités importantes.

D'abord, elles sont conceptuellement simples. Une fois la formule du gradient établie, l'algorithme consiste simplement à générer des épisodes, à calculer des retours G_t , puis à appliquer des mises à jour locales sur les paramètres.

Ensuite, elles sont naturellement adaptées aux politiques stochastiques et aux espaces d'actions continus. Elles évitent donc une partie des difficultés rencontrées par les approches fondées sur un $\arg \max_a q(s, a)$.

Enfin, l'estimateur Monte Carlo obtenu est *non biaisé* au sens suivant : en moyenne, il pointe dans la bonne direction, c'est-à-dire que son espérance coïncide avec le vrai gradient de la politique sous les hypothèses standard du cadre épisodique.

Mais cette approche a aussi un défaut majeur : la variance. Le retour G_t peut fluctuer fortement d'un épisode à l'autre, surtout lorsque les récompenses sont rares, retardées ou bruitées. Deux trajectoires assez proches peuvent conduire à des retours très différents, et donc à des mises à jour très différentes. L'apprentissage devient alors instable, lent et sensible au choix du pas α .

On peut résumer la situation ainsi :

Monte Carlo policy gradient : non biaisé, mais souvent de grande variance.

C'est précisément cette faiblesse qui motive l'introduction, dans la section suivante, de méthodes plus stables comme les approches acteur-critique, où l'on remplace une partie du signal Monte Carlo par une estimation de valeur ou d'avantage.

3.4 Actor Critic Policy Gradient

Principe de la méthode

Les méthodes de type *actor-critic* cherchent à conserver l'idée centrale du gradient de la politique, tout en corrigeant l'un de ses principaux défauts pratiques : la grande variance des mises à jour Monte Carlo.

Dans la méthode REINFORCE, l'acteur est déjà présent : c'est la politique paramétrée π_θ , que l'on cherche à améliorer par gradient. Mais la mise à jour repose sur les retours Monte Carlo G_t , donc sur une information qui n'est disponible qu'après avoir observé toute la suite de la trajectoire à partir de l'instant t . Autrement dit, on attend la fin de l'épisode pour savoir comment corriger l'action choisie au temps t . Cette stratégie est conceptuellement simple, mais elle produit souvent un signal d'apprentissage très bruité.

L'idée des méthodes *actor-critic*, formalisée classiquement par Konda et Tsitsiklis [17], est de garder l'acteur — c'est-à-dire le modèle paramétré de la politique π_θ — mais d'ajouter un second composant, appelé *critique*. Le rôle du critique est de fournir, à chaque étape, une évaluation plus locale de la qualité des décisions prises, sans attendre nécessairement la fin de l'épisode.

Le changement de point de vue est important. Au lieu de mettre à jour la politique seulement après des épisodes complets, comme dans une méthode purement Monte Carlo, on cherche maintenant à mettre à jour l'acteur plus finement, souvent après chaque transition observée. Pour cela, il faut remplacer le retour complet G_t , qui regarde loin dans le futur, par un signal plus immédiatement disponible. C'est précisément ce que fournit le critique.

Le critique peut prendre plusieurs formes. Il peut approximer une valeur d'état $v_\pi(s)$, une valeur d'action $q_\pi(s, a)$, ou directement un avantage $A_\pi(s, a)$. Dans tous les cas, son rôle est analogue : il fournit à l'acteur un signal d'apprentissage moins bruité que le retour Monte Carlo complet. Ce signal peut être, par exemple, une estimation d'avantage $\hat{A}_t$, ou une erreur de différence temporelle δ_t .

On retrouve donc ici une idée déjà familière dans ces notes. Dans RL2, les méthodes TD remplaçaient un retour complet par une cible plus locale construite à partir d'une transition et d'une estimation bootstrap [2, section 2.2]. Les méthodes acteur-critique procèdent dans le même esprit : on n'attend plus l'information Monte Carlo complète, on utilise une estimation de valeur pour approcher ce qu'il reste à gagner à partir de l'état courant. Le critique joue ainsi, pour l'acteur, un rôle analogue à celui d'une méthode de prédiction TD pour une fonction de valeur.

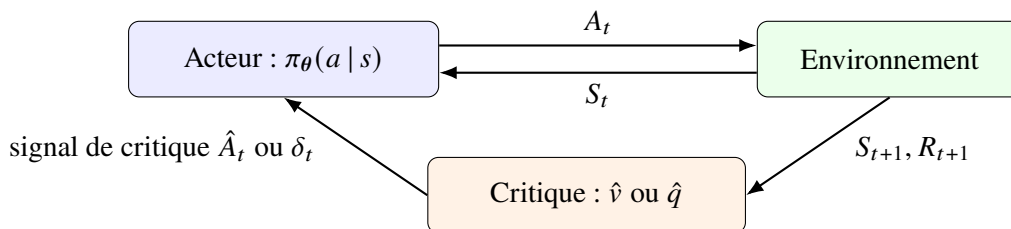


FIGURE 3.2 – Principe général d'une méthode acteur-critique : le critique fournit un signal d'évaluation qui permet de modifier l'acteur sans attendre nécessairement la fin de l'épisode.

La figure 3.2 résume ce mécanisme. L'acteur interagit avec l'environnement en choisissant une action A_t à partir de l'état S_t . L'environnement produit alors l'état suivant S_{t+1} et la récompense R_{t+1} . À partir de cette transition, le critique calcule un signal de correction — par exemple $\hat{A}_t$ ou δ_t — qu'il renvoie à l'acteur. Celui-ci peut alors ajuster immédiatement ses paramètres, sans attendre la fin de l'épisode.

Le principe général est donc le suivant :

- l'*acteur* représente la politique et décide ;
- le *critique* évalue localement la qualité de ce qui vient de se passer ;
- cette évaluation sert à mettre à jour l'acteur plus tôt et avec une variance généralement plus faible que dans une méthode purement Monte Carlo.

Les méthodes acteur-critique combinent ainsi deux idées complémentaires : une optimisation directe de politique du côté de l'acteur, et une estimation de valeur du côté du critique. C'est cette combinaison qui les rend beaucoup plus efficaces en pratique que les méthodes Monte Carlo policy gradient les plus élémentaires.

Policy Gradient theorem

Le résultat théorique central est le *théorème du gradient de la politique* (*policy gradient theorem*), dû en particulier à Sutton, McAllester, Singh et Mansour [16]. Il donne une expression du gradient de la performance sans dériver explicitement la distribution stationnaire induite par la politique.

Pour un critère actualisé, on introduit la mesure d'occupation actualisée

$$\mu_\pi(s) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_\pi(S_t = s).$$

Un mot d'intuition sur la quantité $\mu_\pi(s)$ peut être utile. Pour chaque instant t , le terme $\mathbb{P}_\pi(S_t = s)$ désigne la probabilité d'être dans l'état s au temps t lorsque l'agent suit la politique π . Cette probabilité dépend naturellement de π , puisque la politique influence les actions choisies, donc les états qui seront visités au cours du temps. La quantité $\mu_\pi(s)$ n'est donc pas, en général, une probabilité, mais une *mesure d'occupation actualisée*. Elle additionne les probabilités de visiter l'état s à différents instants, en donnant plus de poids aux visites proches qu'aux visites lointaines. On peut l'interpréter comme une mesure de l'importance de l'état s sous la politique π , du point de vue du critère actualisé. Lorsque $\gamma < 1$, cette quantité reste finie, car elle est bornée par une série géométrique convergente :

$$0 \leq \mu_\pi(s) \leq \sum_{t=0}^{\infty} \gamma^t = \frac{1}{1 - \gamma}.$$

À une constante multiplicative près, le théorème affirme que

$$\nabla_\theta J(\theta) = \sum_{s \in \mathcal{S}} \mu_{\pi_\theta}(s) \sum_{a \in \mathcal{A}} q_{\pi_\theta}(s, a) \nabla_\theta \pi_\theta(a | s).$$

En utilisant l'identité

$$\nabla_\theta \pi_\theta(a | s) = \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s),$$

on obtient l'écriture équivalente

$$\nabla_\theta J(\theta) = \mathbb{E}_{\substack{S \sim \mu_{\pi_\theta} \\ A \sim \pi_\theta(\cdot | S)}} \left[q_{\pi_\theta}(S, A) \nabla_\theta \log \pi_\theta(A | S) \right].$$

Le point essentiel n'est pas la forme exacte de la constante de normalisation, mais le message mathématique : le gradient dépend de la valeur d'action sous la politique courante, pondérée par le score de la politique. Il n'est pas nécessaire de différencier explicitement la dynamique de l'environnement.

Idée de preuve. On part de l'identité de Bellman

$$v_\pi(s) = \sum_a \pi(a | s) q_\pi(s, a),$$

puis on dérive par rapport à θ . Apparaissent alors deux types de termes : ceux qui viennent de la dérivation explicite de la politique, et ceux qui viennent de la dépendance de q_π à la politique elle-même. En développant récursivement cette seconde dépendance à l'aide de Bellman, on obtient une somme sur tous les temps futurs. Cette somme se réorganise précisément en mesure d'occupation des états. Le résultat final fait disparaître les dérivées imbriquées de la distribution des états et laisse la formule ci-dessus. Le mécanisme conceptuel est donc un *télescopage par la dynamique markovienne*.

Action-Value Actor Critic

Une première version naturelle des méthodes acteur-critique consiste à utiliser un critique qui approxime directement la valeur d'action $q_{\pi_{\theta}}(s, a)$. On note alors cette approximation $\hat{q}(s, a, \mathbf{w})$, où $\mathbf{w}$ désigne les paramètres du critique.

L'idée générale est la suivante. On garde l'acteur, c'est-à-dire la politique paramétrée π_{θ} , mais au lieu d'attendre la fin d'un épisode complet pour construire un retour Monte Carlo G_t , on cherche à mettre à jour le système plus tôt, au fil des transitions observées. La méthode devient ainsi *online* au sens suivant : les paramètres sont corrigés après chaque transition, ou après un très petit nombre de transitions, et non plus seulement en fin d'épisode comme dans REINFORCE.

Dans cette architecture, le critique ne fournit pas directement un verdict binaire du type « cette action était bonne » ou « cette action était mauvaise ». Il fournit plus précisément une estimation de la quantité $q_{\pi_{\theta}}(S_t, A_t)$, c'est-à-dire du retour futur espéré si l'on part du couple (S_t, A_t) puis que l'on suit ensuite la politique courante. Cette estimation sert alors de signal pour pondérer la mise à jour de l'acteur. Une écriture naturelle est

$$\theta \leftarrow \theta + \alpha \hat{q}(S_t, A_t, \mathbf{w}) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t).$$

Autrement dit, la politique est modifiée dans la direction suggérée par le gradient, avec une intensité qui dépend de la valeur d'action estimée par le critique.

Il faut toutefois être prudent dans l'interprétation. La quantité $\hat{q}(S_t, A_t, \mathbf{w})$ n'est pas encore un signal centré. Elle mesure un niveau de retour futur espéré, mais elle ne dit pas encore, à elle seule, si l'action est bonne ou mauvaise *relativement à l'état courant*. C'est précisément pour cela que les variantes introduisant une baseline ou un avantage sont souvent plus naturelles : elles fournissent un signal comparatif plus informatif. Dans la présente sous-section, on se limite volontairement au cas le plus direct.

Le critique, de son côté, doit être appris. Comme dans RL2 et RL3, on peut l'ajuster par une méthode de prédiction de valeur, typiquement une méthode TD ou Monte Carlo [2, chapitre 2], [3, chapitre 2]. Si l'on choisit une mise à jour TD à un pas, on peut utiliser la cible

$$Y_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}),$$

où $A_{t+1} \sim \pi_{\theta}(\cdot | S_{t+1})$, avec la convention habituelle $Y_t = R_{t+1}$ si S_{t+1} est terminal. L'erreur TD correspondante est

$$\delta_t = Y_t - \hat{q}(S_t, A_t, \mathbf{w}),$$

et une mise à jour naturelle du critique est

$$\mathbf{w} \leftarrow \mathbf{w} + \beta \delta_t \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w}),$$

où $\beta > 0$ est le pas d'apprentissage du critique.

On obtient ainsi un schéma *online* élémentaire où, à chaque transition, le critique est d'abord corrigé à partir de l'expérience courante, puis l'acteur est ajusté à l'aide du signal fourni par le critique.

Algorithme : Action-Value Actor-Critic (version conceptuelle, online)

1. initialiser les paramètres θ de l'acteur et $\mathbf{w}$ du critique ;
2. observer l'état initial S_0 ;
3. pour $t = 0, 1, 2, \dots$ jusqu'à la fin de l'épisode :
 - (a) choisir une action

$$A_t \sim \pi_{\theta}(\cdot | S_t);$$

- (b) exécuter A_t , puis observer R_{t+1} et S_{t+1} ;

(c) si S_{t+1} n'est pas terminal, échantillonner aussi

$$A_{t+1} \sim \pi_{\theta}(\cdot | S_{t+1});$$

(d) construire la cible TD

$$Y_t = \begin{cases} R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}), & \text{si } S_{t+1} \text{ n'est pas terminal,} \\ R_{t+1}, & \text{sinon;} \end{cases}$$

(e) calculer l'erreur TD

$$\delta_t = Y_t - \hat{q}(S_t, A_t, \mathbf{w});$$

(f) mettre à jour le critique :

$$\mathbf{w} \leftarrow \mathbf{w} + \beta \delta_t \nabla_{\mathbf{w}} \hat{q}(S_t, A_t, \mathbf{w});$$

(g) mettre à jour l'acteur :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \hat{q}(S_t, A_t, \mathbf{w}) \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t | S_t);$$

(h) remplacer S_t par S_{t+1} et continuer.

Ce schéma fait bien apparaître la différence d'esprit avec REINFORCE. Dans une méthode Monte Carlo pure, on doit attendre la fin de l'épisode pour disposer des retours G_t . Ici, au contraire, chaque transition

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

peut déjà être utilisée pour améliorer le critique, puis pour corriger l'acteur. L'information est donc exploitée plus tôt, ce qui rend l'apprentissage plus réactif et, en général, moins bruité.

En résumé, dans un *action-value actor-critic*,

- l'acteur représente la politique $\pi_{\boldsymbol{\theta}}$;
- le critique approxime $q_{\pi_{\boldsymbol{\theta}}}(s, a)$;
- les deux ensembles de paramètres $\boldsymbol{\theta}$ et $\mathbf{w}$ sont mis à jour au fil des transitions ;
- cette mise à jour incrémentale constitue précisément le passage d'une logique Monte Carlo par épisode à une logique online.

Cette version contient déjà l'idée essentielle des méthodes acteur-critique : utiliser une estimation de valeur pour guider plus localement et plus efficacement l'apprentissage de la politique.

Fonctions de valeur compatibles

Dans la théorie classique des méthodes acteur-critique, une idée importante est celle de *fonction de valeur compatible* (*compatible function approximation*) [16]. L'idée n'est plus seulement de choisir un critique raisonnable, mais de choisir un critique dont la forme soit spécialement adaptée à la géométrie du gradient de la politique.

On cherche à approximer la valeur d'action $q_{\pi_{\boldsymbol{\theta}}}(s, a)$ par une fonction paramétrée

$$Q_{\mathbf{w}}(s, a),$$

où $\mathbf{w}$ désigne les paramètres du critique. La notion de compatibilité impose alors une contrainte particulière : les directions de variation du critique par rapport à $\mathbf{w}$ doivent coïncider avec les directions de variation de la politique par rapport à $\boldsymbol{\theta}$. Formellement, on demande

$$\nabla_{\mathbf{w}} Q_{\mathbf{w}}(s, a) = \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a | s).$$

Une façon naturelle de satisfaire cette condition est de prendre un critique linéaire de la forme

$$Q_{\mathbf{w}}(s, a) = \mathbf{w}^\top \nabla_{\theta} \log \pi_{\theta}(a | s).$$

Cette construction peut sembler artificielle au premier abord, mais elle a une conséquence théorique importante. Si, en plus, les paramètres $\mathbf{w}$ sont choisis de façon à approximer au mieux la vraie valeur d'action $q_{\pi_{\theta}}(s, a)$, alors le signal fourni par le critique n'introduit pas de biais supplémentaire dans la formule du gradient.

Plus précisément, on considère l'erreur quadratique moyenne

$$\mathcal{E}(\mathbf{w}) = \mathbb{E}_{\pi_{\theta}} \left[(q_{\pi_{\theta}}(S, A) - Q_{\mathbf{w}}(S, A))^2 \right],$$

où l'espérance est prise sous la distribution on-policy pertinente du théorème du gradient de la politique. On suppose alors que $\mathbf{w}$ minimise cette erreur quadratique moyenne.

On obtient alors le résultat suivant.

Théorème 3.1 (Approximation compatible de la valeur d'action). *Supposons que les deux conditions suivantes soient satisfaites :*

1. la fonction d'approximation $Q_{\mathbf{w}}(s, a)$ est compatible avec la politique, c'est-à-dire

$$\nabla_{\mathbf{w}} Q_{\mathbf{w}}(s, a) = \nabla_{\theta} \log \pi_{\theta}(a | s);$$

2. les paramètres $\mathbf{w}$ minimisent l'erreur quadratique moyenne

$$\mathcal{E}(\mathbf{w}) = \mathbb{E}_{\pi_{\theta}} \left[(q_{\pi_{\theta}}(S, A) - Q_{\mathbf{w}}(S, A))^2 \right].$$

Alors le gradient de la politique peut s'écrire exactement sous la forme

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(A | S) Q_{\mathbf{w}}(S, A) \right].$$

Le point essentiel est donc le suivant : dans ce cadre, on peut remplacer la vraie valeur d'action $q_{\pi_{\theta}}(s, a)$, généralement inconnue, par son approximation compatible $Q_{\mathbf{w}}(s, a)$ sans déformer la formule du gradient. Le critique n'est pas ici un prédicteur arbitraire ; il est construit de façon à être cohérent avec la structure même de l'acteur.

Sur le plan intuitif, cela signifie que le critique ne fournit pas seulement une estimation numérique de la qualité d'une action. Il fournit cette estimation dans les bonnes directions, c'est-à-dire dans les directions qui comptent réellement pour modifier la politique. C'est cette adéquation entre la forme du critique et la forme du gradient de la politique que le mot « compatible » cherche à capturer.

Cette idée joue aussi un rôle important dans le lien avec le *gradient naturel* (*natural gradient*), que l'on ne développera pas ici en détail. Pour le présent cours, il suffit de retenir qu'un critique bien choisi n'est pas seulement plus précis ; il peut aussi être structurellement adapté à la manière dont l'acteur apprend.

Utilisation de Baseline

Une manière très efficace de réduire la variance consiste à soustraire à $q_{\pi}(S_t, A_t)$ ou à G_t une quantité appelée *baseline*. L'idée repose sur une identité très simple.

Propriété 3.1 (Baseline sans biais). Soit $b(s)$ une fonction de l'état seulement. Alors

$$\mathbb{E} \left[b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) \right] = 0.$$

Preuve esquissée. Conditionnons par $S_t = s$. On obtient

$$\begin{aligned}
\mathbb{E}[b(S_t) \nabla_{\theta} \log \pi_{\theta}(A_t | S_t) | S_t = s] &= b(s) \sum_{a \in \mathcal{A}} \pi_{\theta}(a | s) \nabla_{\theta} \log \pi_{\theta}(a | s) \\
&= b(s) \sum_{a \in \mathcal{A}} \nabla_{\theta} \pi_{\theta}(a | s) \\
&= b(s) \nabla_{\theta} \sum_{a \in \mathcal{A}} \pi_{\theta}(a | s) \\
&= b(s) \nabla_{\theta} 1 = 0
\end{aligned}$$

car le gradient d'une fonction constante est nul. Le cas continu est analogue en remplaçant la somme par une intégrale. $\square$

Cette proposition montre que l'on peut remplacer le signal d'apprentissage par

$$q_{\pi}(S_t, A_t) - b(S_t)$$

sans modifier le gradient moyen. Le choix le plus naturel est

$$b(S_t) = v_{\pi}(S_t).$$

On obtient alors l'avantage

$$A_{\pi}(s, a) = q_{\pi}(s, a) - v_{\pi}(s),$$

qui mesure à quel point l'action a est meilleure ou moins bonne que le comportement moyen de la politique dans l'état s .

Estimation de l'avantage

En pratique, on ne connaît ni q_{π} ni v_{π} exactement. Il faut donc estimer l'avantage. Plusieurs possibilités existent.

La plus simple consiste à utiliser un critique d'état $\hat{v}$ et l'erreur TD

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}) - \hat{v}(S_t).$$

Cette quantité peut être vue comme une estimation locale de l'avantage. Intuitivement, si la récompense reçue plus la valeur prédite de l'état suivant est supérieure à la valeur prédite de l'état courant, alors l'action réalisée a été plutôt meilleure que prévu ; on veut donc en augmenter la probabilité.

Une autre possibilité consiste à utiliser des retours sur plusieurs pas, ou des combinaisons pondérées de retours à plusieurs horizons. Cela mène à des estimations d'avantage de type *n-step* ou *generalized advantage estimation* (GAE) [19]. Sans entrer ici dans tous les détails, il faut retenir l'idée de fond : entre Monte Carlo pur et TD pur, on peut construire toute une famille de compromis biais-variance.

Un acteur-critique simple peut donc prendre la forme suivante :

$$\begin{aligned}
\delta_t &= R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w}), \\
\mathbf{w} &\leftarrow \mathbf{w} + \beta \delta_t \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}), \\
\boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \alpha \delta_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(A_t | S_t).
\end{aligned}$$

On reconnaît bien ici l'entrelacement de deux apprentissages : un apprentissage de valeur pour le critique, et un apprentissage de politique pour l'acteur.

Pourquoi cette version est-elle préférable à la version la plus simple ? Dans la version élémentaire de l'acteur-critique, l'acteur était mis à jour à l'aide d'une estimation directe de la valeur d'action $\hat{q}(S_t, A_t, \mathbf{w})$. Cette approche est naturelle, mais elle présente un inconvénient important : le signal fourni à l'acteur n'est pas centré. Une grande valeur de $\hat{q}(S_t, A_t, \mathbf{w})$ peut simplement refléter le fait que l'état S_t est globalement favorable, sans indiquer clairement si l'action A_t est meilleure ou moins bonne que ce que l'on ferait habituellement dans cet état.

L'introduction d'une baseline, puis de l'avantage

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s),$$

corrige précisément ce point. Le signal transmis à l'acteur ne mesure plus un niveau absolu de retour futur espéré, mais un écart relatif : il indique si l'action choisie fait mieux ou moins bien que la référence naturelle donnée par la valeur de l'état. Autrement dit, on ne demande plus seulement : « quel retour futur peut-on espérer ? », mais plutôt : « cette action a-t-elle été meilleure que ce que l'on attendait déjà dans cet état ? »

Ce recentrage présente plusieurs avantages. D'abord, il réduit en général la variance des mises à jour, car une partie des fluctuations communes à toutes les actions d'un même état est soustraite. Ensuite, le signal devient plus informatif pour l'acteur : une estimation d'avantage positive pousse à renforcer l'action choisie, tandis qu'une estimation négative pousse à l'affaiblir relativement. Enfin, cette écriture rapproche naturellement les méthodes acteur-critique modernes de la logique générale

$$\text{mise à jour de l'acteur} \propto \text{signal d'avantage} \times \nabla_{\theta} \log \pi_{\theta}(A_t | S_t),$$

qui constitue aujourd'hui l'une des formes les plus standard du policy gradient en pratique.

En résumé, l'utilisation d'une baseline et d'une estimation de l'avantage ne change pas le problème que l'on cherche à résoudre ; elle change surtout la qualité du signal d'apprentissage transmis à l'acteur. On obtient ainsi des mises à jour plus stables, plus interprétables, et en général plus efficaces que dans la version la plus brute fondée uniquement sur $\hat{q}(S_t, A_t, \mathbf{w})$.

3.5 Variantes / Extensions

Offline Batch TD Actor-Critic

Dans une version *offline batch* d'un acteur-critique, on collecte d'abord un lot de trajectoires ou de transitions, puis l'on met à jour le critique et l'acteur à partir de ce lot. L'avantage de cette procédure est une meilleure stabilité : les gradients de l'acteur sont calculés à partir d'un ensemble de données plus riche, et le critique peut être réentraîné plusieurs fois sur les mêmes données.

L'inconvénient est évident : le lot a été généré par une politique plus ancienne. Si l'acteur change trop vite, les données deviennent rapidement moins représentatives de la politique courante. Il faut alors introduire des mécanismes correctifs ou limiter l'amplitude des mises à jour.

Online TD Actor-Critic

À l'autre extrême, l'acteur-critique *online* met à jour l'acteur et le critique presque à chaque transition. Cette version est plus réactive, consomme moins de mémoire et s'intègre naturellement à des flux continus d'interaction. Elle est aussi plus proche de l'esprit du TD(0) étudié dans RL2 [2, section 2.2].

Mais cette réactivité a un prix : les données consécutives sont fortement corrélées. Or les méthodes d'optimisation stochastique sont plus confortables lorsque les échantillons sont peu corrélés. C'est l'une des raisons pour lesquelles les implémentations modernes utilisent souvent des mini-batches et divers mécanismes de décorrélation.

Online AC : corrélations, versions synchronisées et asynchrones

Dans un acteur-critique purement online, les mises à jour peuvent être effectuées après chaque transition observée. Cette idée est séduisante, car elle permet d'apprendre en continu, sans attendre la fin d'un épisode. Mais elle fait apparaître un problème classique : deux transitions consécutives d'une même trajectoire se ressemblent souvent beaucoup. Les états successifs sont proches, les actions peuvent être voisines, et les cibles TD sont elles-mêmes fortement corrélées. Le bruit des mises à jour n'est donc pas un bruit indépendant ; il possède une structure temporelle qui peut ralentir l'apprentissage, voire le rendre instable.

Une idée importante a consisté à faire interagir plusieurs acteurs avec plusieurs copies de l'environnement en parallèle. Chaque acteur collecte sa propre expérience, ce qui permet d'obtenir un flux de données plus varié que dans une unique trajectoire. C'est l'esprit des méthodes parallèles acteur-critique, en particulier A3C (*asynchronous advantage actor-critic*) [18] et sa version synchronisée, souvent appelée A2C.

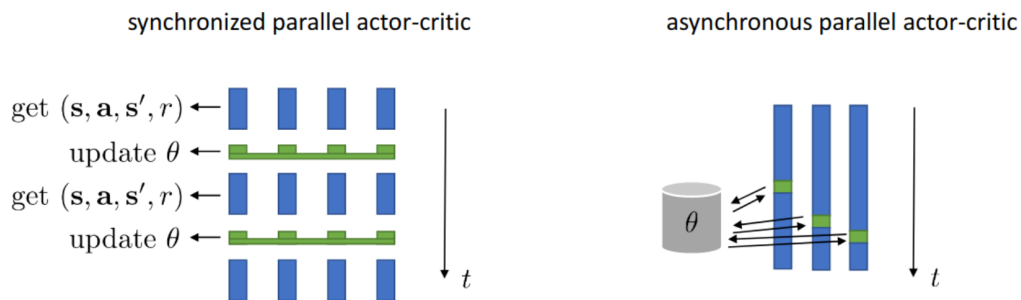


FIGURE 3.3 – Principe des versions parallèles de l'acteur-critique. À gauche : version synchronisée, où plusieurs acteurs collectent d'abord des transitions puis une mise à jour commune est effectuée. À droite : version asynchrone, où plusieurs travailleurs interagissent et mettent à jour des paramètres partagés à des instants différents.

La figure 3.3 donne une image concrète de ces deux variantes.

Dans la partie gauche, on voit le fonctionnement *synchronisé*. Plusieurs acteurs interagissent chacun avec leur environnement pendant un petit nombre d'étapes. On recueille donc plusieurs transitions du type

$$(s, a, s', r),$$

provenant de plusieurs copies de l'environnement. Une fois cette collecte terminée, on effectue une mise à jour commune des paramètres θ et éventuellement du critique. Puis tous les acteurs repartent à partir de ces nouveaux paramètres. Le processus alterne donc entre deux phases bien séparées :

- collecte parallèle d'expérience ;
- mise à jour synchronisée des paramètres.

Cette organisation est simple à analyser et bien adaptée au calcul vectorisé sur machine moderne.

Dans la partie droite, on voit le fonctionnement *asynchrone*. Les différents travailleurs n'attendent pas les uns les autres. Chacun interagit avec sa copie de l'environnement à son propre rythme, calcule un gradient ou une correction locale, puis l'applique presque immédiatement à un ensemble de paramètres partagés. Les mises à jour arrivent donc à des instants différents, sans barrière de synchronisation globale. L'idée est de gagner en réactivité et en débit de calcul, au prix d'une analyse plus délicate, car certains travailleurs peuvent calculer leurs corrections à partir de paramètres déjà légèrement anciens.

Ces deux variantes partagent la même intuition de fond :

- accélérer la collecte de données en utilisant plusieurs environnements en parallèle ;
- réduire les corrélations temporelles en mélangeant des trajectoires issues de plusieurs acteurs ;
- conserver la structure acteur-critique, avec un acteur mis à jour à partir d'un signal d'avantage ou d'erreur TD fourni par un critique.

La différence principale porte donc sur l'organisation des mises à jour. Dans une méthode synchronisée, on attend que tous les acteurs aient terminé leur portion de travail avant de mettre à jour les paramètres. Dans une méthode asynchrone, chaque acteur pousse ses corrections dès qu'elles sont disponibles. Conceptuellement, les deux approches restent des méthodes acteur-critique ; elles diffèrent surtout par leur manière de paralléliser l'apprentissage.

Soft Actor-Critic / Soft RL

Jusqu'ici, la plupart des méthodes étudiées cherchent, explicitement ou implicitement, à concentrer progressivement la politique sur les actions jugées les meilleures. Dans un problème tabulaire, c'est très visible avec des règles gloutonnes ou ε -gloutonnes : à mesure que l'apprentissage avance, la politique tend à devenir presque déterministe. Mais cette tendance apparaît aussi dans les méthodes de gradient de politique et dans les méthodes acteur-critique : lorsqu'une direction d'action semble meilleure que les autres, l'algorithme augmente sa probabilité et finit souvent par privilégier très fortement cette seule action.

Cette évolution vers une politique presque déterministe n'est pas toujours souhaitable. Elle peut conduire à un phénomène de « verrouillage » prématuré : dès qu'un comportement semble localement bon, l'algorithme cesse trop vite d'explorer des alternatives qui pourraient pourtant conduire à de meilleures récompenses à long terme. Autrement dit, une politique déterministe n'est pas nécessairement la meilleure politique, surtout dans des problèmes où l'exploration reste importante pendant l'apprentissage.

L'idée de *Soft RL*, ou plus précisément de *maximum entropy reinforcement learning*, consiste à corriger cette tendance en ajoutant à l'objectif un terme qui favorise les politiques suffisamment entropiques. On ne cherche donc plus seulement à maximiser la somme des récompenses, mais un compromis entre

- obtenir de bonnes récompenses ;
- conserver une politique assez diversifiée pour continuer à explorer.

Pour une politique stochastique $\pi(\cdot | s)$, on définit son entropie par

$$\mathcal{H}(\pi(\cdot | s)) = -\mathbb{E}_{A \sim \pi(\cdot | s)} [\log \pi(A | s)].$$

Cette quantité est grande lorsque la politique est dispersée entre plusieurs actions, et petite lorsqu'elle est très concentrée, donc presque déterministe.

On remplace alors l'objectif classique par un objectif *soft* du type

$$J_{\text{soft}}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t \geq 0} \gamma^t \left(R_{t+1} + \alpha \mathcal{H}(\pi(\cdot | S_t)) \right) \right],$$

où $\alpha > 0$ règle le compromis entre récompense et entropie. Si α est petit, on se rapproche du RL standard ; si α est plus grand, on encourage davantage la politique à rester aléatoire.

L'effet intuitif est important. Dans le cadre standard, une politique est récompensée uniquement pour les gains qu'elle obtient. Dans le cadre *soft*, elle est aussi récompensée pour le fait de ne pas devenir trop confiante trop vite. L'entropie joue donc comme un terme de régularisation qui pousse la politique à conserver plusieurs actions plausibles, au lieu de s'effondrer prématurément vers un comportement déterministe.

Cette modification change aussi la manière dont on écrit les équations de Bellman. Sans entrer dans tous les détails, l'idée générale est que le terme d'entropie vient s'ajouter au critère futur. Dans une écriture de type valeur d'action, on obtient une relation de la forme

$$q_{\pi}^{\text{soft}}(s, a) = \mathbb{E} \left[R_{t+1} + \gamma (q_{\pi}^{\text{soft}}(S_{t+1}, A_{t+1}) - \alpha \log \pi(A_{t+1} | S_{t+1})) \mid S_t = s, A_t = a \right].$$

Le terme

$$-\alpha \log \pi(A_{t+1} | S_{t+1})$$

reflète précisément la contribution de l'entropie. On voit donc que, dans le cadre *soft*, la valeur future d'une politique ne dépend plus seulement des récompenses attendues, mais aussi de son degré de diversification.

Une autre manière de lire ce principe est la suivante : dans chaque état, la politique n'est plus poussée vers l'action qui maximise brutalement Q , mais vers une distribution qui favorise les grandes valeurs de Q tout en restant entropique. Cela peut se formaliser par une mise à jour de politique liée à une minimisation de divergence de Kullback–Leibler vers une loi de type Boltzmann, proportionnelle à

$$\exp\left(\frac{Q(s, \cdot)}{\alpha}\right).$$

Nous n'entrerons pas ici dans le détail de cette dérivation, mais il est utile d'en retenir le message : dans le cadre *soft*, la politique cible n'est plus une règle purement gloutonne, mais une distribution adoucie par l'entropie.

Soft Actor-Critic (SAC) [21] est une famille très influente de méthodes qui combine cette idée d'entropie maximale avec une architecture acteur-critique moderne, généralement en mode off-policy. Le critique apprend des quantités de type Q dans le cadre *soft*, tandis que l'acteur apprend une politique stochastique qui cherche à la fois de bonnes récompenses et une entropie suffisante.

Il existe de nombreuses variantes de SAC dans la littérature. Certaines versions utilisent explicitement une fonction de valeur d'état V , d'autres travaillent essentiellement avec une ou plusieurs fonctions Q , et d'autres encore utilisent des reformulations en termes d'avantage. Les versions modernes les plus courantes n'utilisent d'ailleurs pas toujours une fonction V séparée. Ces différences sont importantes en pratique, mais elles ne changent pas l'idée conceptuelle de base.

Pour le présent cours, il faut surtout retenir les trois messages suivants :

- les méthodes classiques tendent souvent à rendre la politique rapidement presque déterministe ;
- le cadre *soft* ajoute un terme d'entropie à l'objectif pour encourager une exploration durable ;
- *Soft Actor-Critic* applique cette idée dans une architecture acteur-critique moderne, particulièrement efficace en contrôle continu.

Autrement dit, SAC ne change pas seulement un détail technique de l'acteur-critique. Il modifie le problème lui-même : on ne cherche plus simplement une politique qui maximise la récompense, mais une politique qui maximise la récompense tout en restant suffisamment entropique.

Conclusion

Le gradient de la politique marque un changement important dans la manière d’aborder le RL. Dans les méthodes basées sur la valeur, le problème central était d’apprendre *combien vaut* une décision. Ici, le problème devient : *comment modifier directement la loi de décision pour améliorer la performance moyenne ?*

L’astuce de la fonction de score fournit la réponse mathématique de base. Elle mène à une formule générale du gradient, indépendante du modèle de l’environnement. À partir de là, deux grandes familles apparaissent naturellement. D’une part, les méthodes Monte Carlo comme REINFORCE, simples mais de forte variance. D’autre part, les méthodes acteur-critique, qui ajoutent un estimateur de valeur pour construire un signal d’apprentissage plus stable.

Sur le plan moderne, beaucoup d’algorithmes importants se rangent dans cette lignée : A2C, A3C, PPO, SAC et d’autres encore. Le vocabulaire peut varier, les raffinements techniques sont nombreux, mais le coeur conceptuel reste celui étudié ici : paramétrer une politique, calculer une direction d’amélioration, et utiliser l’expérience pour ajuster ses paramètres.

Bibliographie

- [1] Marin Ferecatu, *Apprentissage par renforcement : cadre général et programmation dynamique*, notes de cours RL1, UE RCP211, LeCNAM, Paris, 2026.
- [2] Marin Ferecatu, *Apprentissage par renforcement : prédiction et contrôle Model Free*, notes de cours RL2, UE RCP211, LeCNAM, Paris, 2026.
- [3] Marin Ferecatu, *Approximation de la fonction de valeur et réseaux DQN*, notes de cours RL3, UE RCP211, LeCNAM, Paris, 2026.
- [4] Richard S. Sutton and Andrew G. Barto, *Reinforcement Learning : An Introduction*, 2nd edition, The MIT Press, 2018.
<http://incompleteideas.net/book/the-book-2nd.html> (Licence : Creative Commons Attribution-NonCommercial-NoDerivs 2.0 Generic.)
- [5] Shiyu Zhao, *Mathematical Foundations of Reinforcement Learning*, Springer, 2026.
Projet associé : <https://github.com/MathFoundationRL/Book-Mathematical-Foundation-of-Reinforcement-Learning>
- [6] David Silver, *Lectures on Reinforcement Learning*, 2015.
<https://www.davidsilver.uk/teaching/>
- [7] Gerald Tesauro, “Temporal Difference Learning and TD-Gammon”, *Communications of the ACM*, 38(3), 1995, p. 58–68.
- [8] Christopher J. C. H. Watkins and Peter Dayan, “Q-learning”, *Machine Learning*, 8(3–4), 1992, p. 279–292.
- [9] Volodymyr Mnih et al., “Human-level control through deep reinforcement learning”, *Nature*, 518, 2015, p. 529–533.
- [10] Hado van Hasselt, Arthur Guez and David Silver, “Deep Reinforcement Learning with Double Q-learning”, *Proceedings of AAAI*, 2016.
- [11] Ziyu Wang et al., “Dueling Network Architectures for Deep Reinforcement Learning”, *Proceedings of ICML*, 2016.
- [12] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar et D. Silver, “Rainbow : Combining Improvements in Deep Reinforcement Learning”, *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [13] W. Dabney, G. Ostrovski, D. Silver et R. Munos, “Implicit Quantile Networks for Distributional Reinforcement Learning”, *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [14] Tom Schaul, John Quan, Ioannis Antonoglou and David Silver, “Prioritized Experience Replay”, *Proceedings of ICLR*, 2016.
- [15] Ronald J. Williams, “Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning”, *Machine Learning*, 8, 1992, p. 229–256.
- [16] Richard S. Sutton, David McAllester, Satinder Singh and Yishay Mansour, “Policy Gradient Methods for Reinforcement Learning with Function Approximation”, *Advances in Neural Information Processing Systems*, 12, 2000.

- [17] Vijay R. Konda and John N. Tsitsiklis, “Actor-Critic Algorithms”, *Advances in Neural Information Processing Systems*, 12, 2000.
- [18] Volodymyr Mnih et al., “Asynchronous Methods for Deep Reinforcement Learning”, *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- [19] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan and Pieter Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation”, *Proceedings of ICLR*, 2016.
- [20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford and Oleg Klimov, “Proximal Policy Optimization Algorithms”, arXiv :1707.06347, 2017.
- [21] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel and Sergey Levine, “Soft Actor-Critic : Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”, *Proceedings of the 35th International Conference on Machine Learning*, 2018.
- [22] Reuven Y. Rubinstein and Dirk P. Kroese, *The Cross-Entropy Method : A Unified Approach to Combinatorial Optimization, Monte-Carlo Simulation, and Machine Learning*, Springer, 2004.
- [23] N. Kohl and P. Stone, *Policy Gradient Reinforcement Learning for Fast Quadrupedal Locomotion*, Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 2004, pp. 2619–2624.
- [24] N. H. Siboni, *On the “Causality” Step in Policy Gradient Derivations : A Pedagogical Reconciliation of Full Return and Reward-to-Go*, arXiv preprint, 2026.